UNIVERSITÉ LAVAL

Rapport - Livrable 3

Design III

présenté à

Denis Laurendeau, Dominic Grenier, Amine Miled & François Pomerleau,

par

Équipe 8

matricule	*nom*
111 188 201	Bourbonnais-Roy, Sophie
111 141 291	Douch, Ayoub
111 221 913	Drolet, Dominic
111 181 978	Elfakrani, Rochdi
111 130 199	Lacroix, François-Joseph
111 183 497	Marois-Boucher, Vincent
111 187 837	Noël-Giroux, Nicolas
111 182 095	Soulard, Guillaume
111 178 727	St-Pierre, Gabriel

Université Laval

19 avril 2020

Table des matières

Table des figures v

Liste des tableaux vi

1 Introduction 1

2 Analyse préliminaire 2
2.1 Familiarisation avec les équipements . . . 2
2.1.1 Structure mécanique, roues et moteurs . . . 2
2.1.2 Communication sans fil . . . 3
2.1.3 Alimentation . . . 3
2.1.4 Vision artificielle . . . 3
2.1.5 Raspberry Pi . . . 4
2.1.6 Station de base . . . 4
2.1.7 Table de jeu . . . 4
2.1.8 Station de lecture des résistances . . . 4
2.1.9 La zone des rondelles . . . 4
2.1.10 Les rondelles . . . 4
2.1.11 Zone cible . . . 4
2.1.12 Les obstacles . . . 5
2.1.13 Les tableaux de commandes . . . 5
2.2 Expériences préliminaires . . . 6
2.2.1 Communication Wi-Fi . . . 6
2.2.2 Test de la librairie OpenCV sur le raspberry Pi . . . 6
2.2.3 Préhenseur . . . 8
2.2.4 Pont H . . . 9
2.2.5 Alimentation . . . 9
2.2.6 Caméras . . . 10
2.2.7 Asservissement moteur . . . 10
2.2.8 Mesure des courants . . . 11
2.3 Planification du développement logiciel . . . 12
2.3.1 Système de contrôle de versions . . . 12

2.3.2 Gestion de projet sur GitHub . . . 12
2.3.2.1 « Milestones » . . . 12
2.3.2.2 « Issues » . . . 12
2.3.2.3 Kanban . . . 13
2.3.3 Méthodes de travail . . . 13
2.3.3.1 Tests . . . 13
2.3.3.2 Revues de code . . . 14
2.3.3.3 Gestion des branches . . . 14
2.3.4 Gestion des fichiers . . . 14
2.3.5 Langages de programmation utilisés . . . 15
2.3.6 Règles de codage . . . 15
2.3.7 Bibliothèques externes . . . 15

3 Design et implémentation **16**
3.1 Avancement de la construction . . . 16
3.1.1 Pratiques de développement et outils logiciels . . . 16
3.1.2 Séquençage des actions . . . 16
3.1.3 Interface utilisateur . . . 17
3.1.4 Communication entre la station de base et le robot . . . 17
3.1.5 Communication entre le RaspberryPi et le STM32 . . . 17
3.1.6 Vision et reconnaissance . . . 18
3.1.6.1 Caméra monde . . . 18
3.1.6.2 Caméra robot . . . 24
3.1.6.3 Détection des tableaux de commandes (flèches) . . . 24
3.1.7 Planification de trajectoire . . . 29
3.1.8 Alimentation . . . 29
3.1.8.1 Description des composantes . . . 29
3.1.8.2 Solutions retenues et considérées . . . 30
3.1.8.3 Moyens techniques . . . 30
3.1.9 Préhenseur . . . 30
3.1.9.1 Description des composantes . . . 30
3.1.9.2 Solutions retenues et considérées . . . 31
3.1.9.3 Moyens techniques . . . 31
3.1.10 Mesure de resistance . . . 31
3.1.10.1 Description des composantes . . . 31
3.1.10.2 Solutions retenues et considérées . . . 31
3.1.10.3 Moyens techniques . . . 32
3.1.11 Aquisition . . . 35
3.1.11.1 Description des composantes . . . 35
3.1.11.2 Solutions retenues et considérées . . . 35
3.1.11.3 Moyens techniques . . . 35
3.1.12 Traction . . . 35
3.1.12.1 Description des composantes . . . 35

3.1.12.2 Solutions retenues et considérées . . . 35
3.1.12.3 Moyens techniques . . . 35
3.2 Tests unitaires . . . 36
3.2.1 Résultats des tests . . . 36
3.2.2 Résultats de couverture des tests . . . 38

4 Rétrospectives 40
4.1 Gestion du contenu . . . 41
4.1.1 Progrès . . . 41
4.1.1.1 Calibration de la caméra monde et détection des obstacles . . . 41
4.1.1.2 Pathfinding et systèmes de coordonnées . . . 43
4.1.1.3 Détection des tableaux de commandes avec lettres . . . 47
4.1.1.4 PCB . . . 48
4.1.1.5 Maintien de l'information et diffusion des changements . . . 49
4.1.1.6 Interface . . . 49
4.1.2 Éléments manquants . . . 52
4.1.2.1 Détection du robot . . . 52
4.1.2.2 Système de changement de coordonnées complet . . . 52
4.1.2.3 Pathfinding - *padding* ajusté à la rotation du robot . . . 52
4.1.2.4 Connexion de l'interface à la station de base . . . 53
4.1.2.5 Détection des tableaux de commandes . . . 53
4.1.2.6 PCB . . . 53
4.1.2.7 Préhenseur . . . 53
4.1.2.8 Code de vérification pour communication UART . . . 54
4.1.3 Points forts . . . 54
4.1.3.1 Intégration rapide grâce à l'injection automatisée . . . 54
4.1.3.2 Simplicité du système de séquençage . . . 54
4.1.3.3 Architecture bien définie . . . 55
4.1.3.4 Architecture du style de l'interface . . . 55
4.1.3.5 Pathfinding - création de la grille très efficace . . . 55
4.1.4 Points faibles . . . 55
4.1.4.1 Manque de tests automatisés . . . 55
4.2 Gestion des finances . . . 57
4.2.1 Liste des achats . . . 57
4.2.2 Projection des dépenses . . . 57
4.3 Gestion des ressources humaines . . . 58
4.3.1 Outils et protocoles de communication . . . 58
4.3.2 Critiques sur les méthodes de fonctionnement de l'équipe . . . 58

A Plans PCB 59

B Montage préhenseur 63

Table des figures

2.1 Montage mécanique de base pour le test des roues 3
2.2 Test de communication entre la station de base et le Raspberry pi 6
2.3 Image originale 7
2.4 Masque appliqué 8
2.5 Image filtrée 8
2.6 Une branche de l'encodeur à sa fréquence maximale 9
2.7 Capture d'écran de la vision de la caméra monde 10
2.8 Réponse en vitesse des quatre moteurs 11

3.1 Test de la calibration de la caméra 19
3.2 Mauvaise calibration de la caméra 20
3.3 Résultat de la détection des marqueurs Aruco 21
3.4 Image original 22
3.5 Image saturation augmentée et ouverture appliquée 22
3.6 Détection des contours 23
3.7 Rondelles identifié 23
3.8 Tableaux de commandes vus par la caméra du robot 24
3.9 Représentation du format de couleurs HSV 25
3.10 Résultat de la fonction inRange 25
3.11 Première ouverture 25
3.12 Fonction d'érosion 25
3.13 Deuxième ouverture 25
3.14 Résultat de l'addition du masque et de l'image originale. 26
3.15 Panneaux en noir et blanc 26
3.16 Distribution de la corrélation avec la flèche du sens horaire et l'image de base. 27
3.17 Prédiction du positionnement des flèches dans le tableau de commandes. . . 28
3.18 Circuit de contrôle de l'électroaimant du prehenseur 32
3.19 Circuit de lecture de résistance avec lecture de tension 33
3.20 Circuit de lecture de résistance avec mesure de temps 34
3.21 Schéma de l'algorithme de contrôle moteur 36

4.1 Détection améliorée de la base des piliers 42

4.2 Comparaison des couvertures avec ajout de la moitié de la précision au rayon versus sans ajout 44
4.3 Taux de décroissance selon les ordre 1, 2, 3 et 10 45
4.4 Exemple de grille de recherche et chemin trouvé 46
4.5 Temps d'exécutions des étapes de recherche du meilleur chemin 47
4.6 Identification d'un panneau de lettres 48
4.7 Interface améliorée 50

Liste des tableaux

2.2 Identifiants GitHub des membres de l'équipe 12

3.1 Mesure de différentes valeurs sur toute la plage à l'aide du circuit RC 34

4.1 Liste des achats effectués pour la conception du projet 57

Chapitre 1

Introduction

Dans le cadre du cours Design 3, nous avons été chargés du projet iRondelle2020. Ce projet consiste à concevoir un robot entièrement autonome qui aura pour but d'aller chercher trois rondelles. Le choix des rondelles à saisir est dicté par le code de couleur d'une résistance se trouvant à la station de lecture des résistances. Le robot devra par la suite aller déposer ces rondelles à un endroit précis nommé la zone des rondelles dans un ordre lui aussi déterminé par des tableaux de commandes disposés verticalement sur la table de jeu et par le code de couleur de la résistance. Il devra, finalement, faire tout cela en évitant des obstacles.

Les différentes contraintes, fonctionnalités, la planification et l'analyse préliminaire du projet sont détaillés dans les prochaines pages. De plus, les différents choix de design et d'implémentation sont présentés par la suite.

Chapitre 2

Analyse préliminaire

2.1 Familiarisation avec les équipements

2.1.1 Structure mécanique, roues et moteurs

Les composantes mécaniques du robot ont été assemblées de diverses façons afin de déterminer la meilleure disposition pour les composantes déjà choisies telles que la batterie, le pont H, le Raspberry Pi, et la carte de développement pour STM32 (figure 2.1). Les différentes plaques de bois permettent une grande liberté d'ajustement des étages.

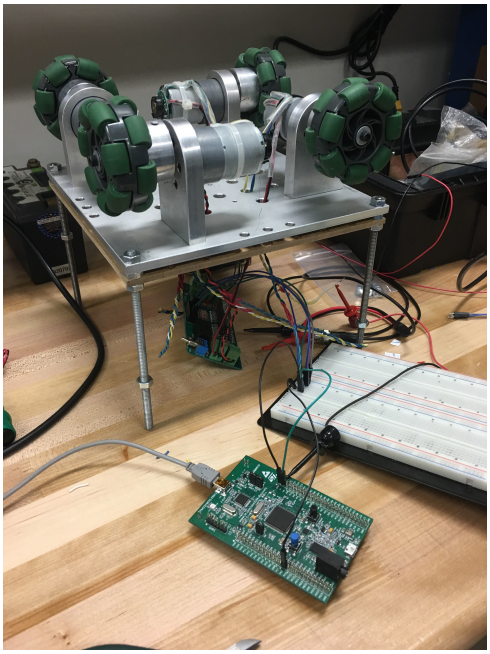


FIGURE 2.1 – Montage mécanique de base pour le test des roues

Les roues ont été assemblées sur la base et chacun des moteurs a été testé avec la source disponible au laboratoire pour valider leur fonctionnement. Les encodeurs, quant à eux, ont été testés avec la source du laboratoire et l'oscilloscope.

2.1.2 Communication sans fil

La communication entre la station de base et le robot se fait par le Raspberry Pi via le réseau Wi-Fi des locaux disponibles pour le projet.

2.1.3 Alimentation

La batterie choisie ne nécessite pas de chargeur spécialisé. Son fonctionnement a été validé en effectuant un cycle charge/décharge dans une charge résistive.

2.1.4 Vision artificielle

La caméra monde sera branchée sur la station de base afin de fournir l'image globale de la table de jeu. Quant à la caméra embarquée, celle-ci sera branchée sur le Raspberry Pi afin de faire la détection des feuilles contenant les tableaux de commande pour déposer les rondelles et possiblement aider à la détection d'obstacles.

2.1.5 Raspberry Pi

Le Raspberry Pi a été installé convenablement sur le robot. Sa fonction est d'effectuer la communication entre le robot et la station de base, en plus d'effectuer certaines actions de façon autonome comme prendre et déposer les rondelles.

2.1.6 Station de base

La station de base affichera toutes les informations demandées et agit en tant que cerveau principal de l'opération. Elle reçoit des données utiles des caméras et du robot afin de déterminer les actions à effectuer et envoie les commandes nécéssaires au robot, comme un déplacement, une rotation de la caméra, etc.

2.1.7 Table de jeu

La table de jeu sur laquelle aura lieu le test du prototype, a des dimensions de $111cm$ de largeur et $231cm$ de longueur. Les différents éléments présents sur la table de jeu sont : une station de lecture des résistances, une zone de rondelles, une zone cible, des obstacles, et des tableaux de commande. Une caméra permet d'obtenir une vue de haut de la table de jeu.

2.1.8 Station de lecture des résistances

La station de lecture des résistances est située sur le mur nord du coin nord-est de l'espace de jeu. Le relais de la station de lecture doit être relié à la masse afin de permttre au dispositif de déterminer la valeur de la résistance à l'aide de la plaquette fourni par le département.

2.1.9 La zone des rondelles

La zone des rondelles, en forme de U, est située sur la largeur complète du mur ouest, et une partie des côtés sud et nord.

2.1.10 Les rondelles

Les rondelles ont un diamètre extérieur de $7.6cm$ et un diamètre de $3.8cm$ pour le disque métallique interne. Ces dernières sont situées à $10cm$ des murs de la table de jeu dans la zone des rondelles. Le poids d'une rondelle est d'environs $31g$. Toutes ces caractéristiques sont importantes pour le design du préhenseur qui devra soulever une rondelle.

2.1.11 Zone cible

La zone cible est un carré vert de côté extérieur de $66cm$. La largeur des côtés du carré sont de $3.1cm$. La zone cible est située du côté est de la table de jeu.

2.1.12 Les obstacles

Les obstacles sont de forme cylindrique et de couleur noire. Leurs dimensions sont les suivantes : 41cm de hauteur totale, 12.5cm pour la partie inférieure et supérieure, et 11.5cm pour la partie du milieu. Il est possible de placer un marqueur sur la partie supérieur du cylindre, un cercle de 12.5cm de diamètre vu de haut.

2.1.13 Les tableaux de commandes

Il y a deux tableaux distincts situés sur le mur ouest de la table de jeu. Chaque tableau est un carré de 15.24cm de côté et possède 9 cases distinctes. Les symboles indiquant le sens de dépôt des rondelles ont une hauteur et une largeur de 4cm.

2.2 Expériences préliminaires

2.2.1 Communication Wi-Fi

Le Rasberry Pi situé sur le robot doit pouvoir communiquer avec une station de base qui lui donne des instructions. Afin de tester cette communication, un programme Python simple a été éxécuté sur le Rasberry Pi à partir de la station de base, en passant par le réseau Wi-Fi du local selon le protocole SSH.

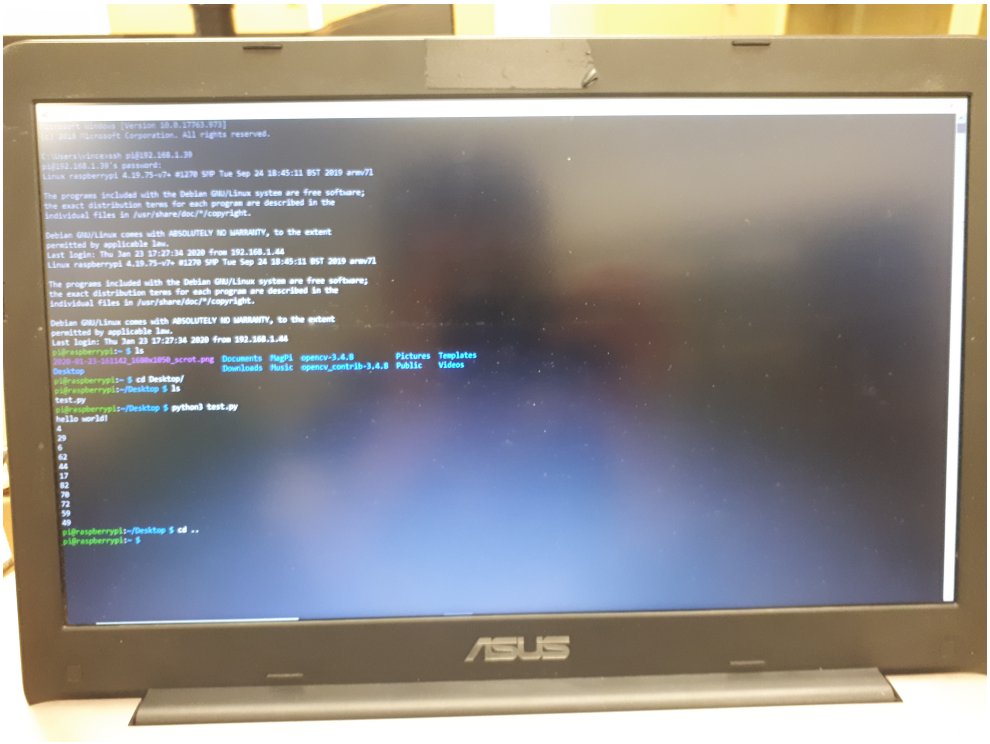


FIGURE 2.2 – Test de communication entre la station de base et le Raspberry pi

2.2.2 Test de la librairie OpenCV sur le raspberry Pi

Le raspberry Pi devra également traiter les images provenant de la caméra embarquée, surtout pour lire les deux tableaux de commande et éviter les obstacles. Ainsi, il est nécessaire de tester la librairie OpenCV. Nous avons utilisé un petit code qui permet d'appliquer un masque (figure 2.4) à la figure 2.3 et de n'afficher que seulement ce qui a la couleur bleue. On obtient ainsi la figure 2.5.

Listing 2.1 – Code utilisé pour le test

```
import numpy as np
import cv2

img2=cv2.imread('/Users/ayoubdouch/Desktopdownload.png',1)
hsv=cv2.cvtColor(img2,cv2.COLOR_BGR2HSV)
lower_red=np.array([110,50,50])
upper_red=np.array([130,255,255])

mask=cv2.inRange(hsv,lower_red,upper_red)

res=cv2.bitwise_and(img2,img2,mask= mask)
cv2.imshow('duck couleur',img2)
cv2.imshow('mask',mask)
cv2.imshow('res',res)
cv2.waitKey(0)
```



FIGURE 2.3 – Image originale



FIGURE 2.4 – Masque appliqué

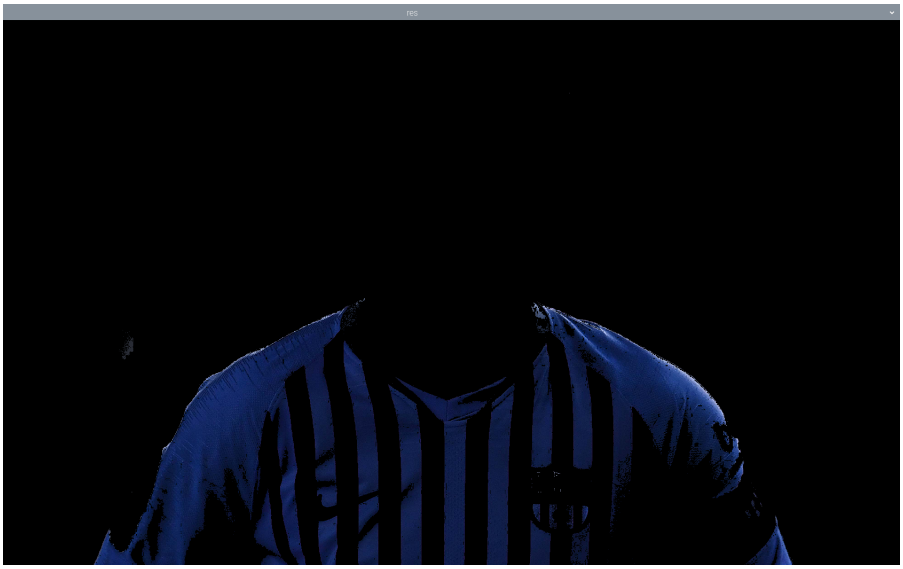


FIGURE 2.5 – Image filtrée

2.2.3 Préhenseur

Afin de récupérer les rondelles, le robot devra être équipé d'un préhenseur. De nombreuses méthodes sont possibles pour d'effectuer la tâche, telles qu'un électroaimant, une pince, une ventouse ou même une pelle. Cependant, chacune de ces méthodes a des avantages et des inconvénients. Utiliser un électroaimant risque de demander davantage de puissance qu'une simple pelle. Par contre, il sera beaucoup plus difficile de concevoir une pelle assez efficace pour

prendre la rondelle, mais sans l'appuyer contre le mur. Il est donc important de bien penser à quelle méthode demeure la plus fiable, simple et efficace. Les rondelles ont été mesurées et analysées afin de diriger ce choix.

2.2.4 Pont H

Le pont H a été testé avec un code de test sur une carte de développement. Ceci a permis de tester le fonctionnement du circuit de contrôle à $3.3V$ (puisque la fiche technique demande $5V$), et aussi de valider la compréhension de son fonctionnement au niveau des freins, du changement de direction et du contrôle de vitesse. Le circuit d'encodeur a également été analysé (figure 2.6) pour le comportement lors du changement de direction et aussi pour valider la fréquence lorsque le moteur tourne à sa vitesse maximale ($\approx 3kHz$), puisque cette fréquence a un impact sur la puissance de calcul nécessaire pour aquisitionner celui-ci. En effectuant ces tests, une grande différence dans la réponse des moteurs a été observée. Finalement, les moteurs ont été essayés en contact avec le sol pour voir à quel point il y a du glissement. Le glissement rend plus difficile le contrôle en position.

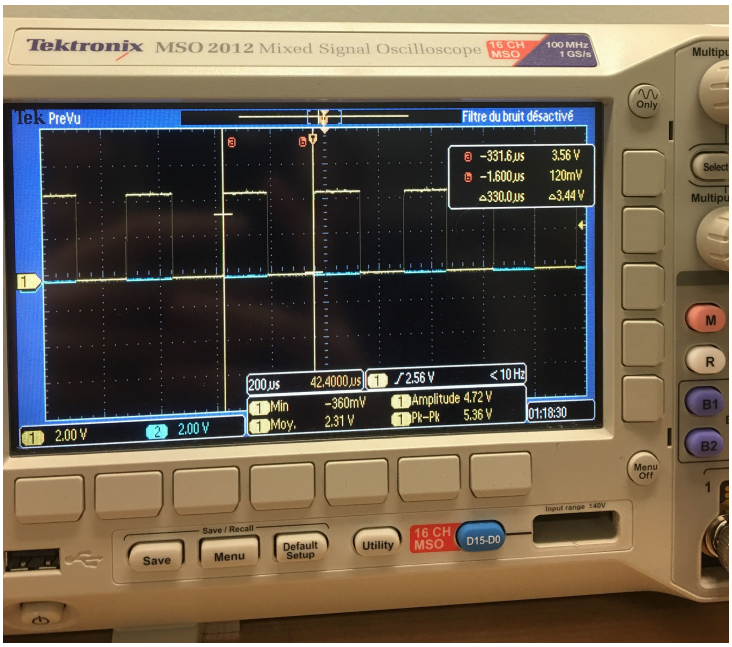


FIGURE 2.6 – Une branche de l'encodeur à sa fréquence maximale

2.2.5 Alimentation

Le pont H a été testé sur toute la plage de la batterie, afin de valider si un régulateur supplémentaire est nécessaire pour ce système.

2.2.6 Caméras

La caméra monde a été testée en utilisant le logiciel *VLC media player* en flux video continu à partir de la station de base. La caméra embarquée a été testée similairement, mais à partir de *VLC media player* sur le Raspberry Pi en flux vidéo continu.

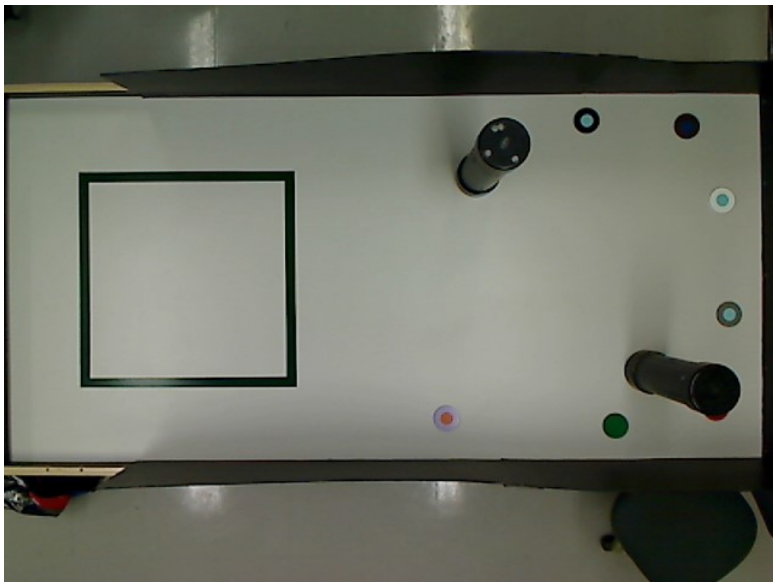


FIGURE 2.7 – Capture d'écran de la vision de la caméra monde

2.2.7 Asservissement moteur

La réponse en vitesse des moteurs a été relevée à l'aide des encodeurs. Il est possible de voir les différences entre les quatre moteurs sur la figure 2.8. Il y a une différence dans le régime permanent des moteurs. La constante de temps des moteurs est sensiblement la même, d'autres expériences devront être faites pour voir si une telle différence pose problème.

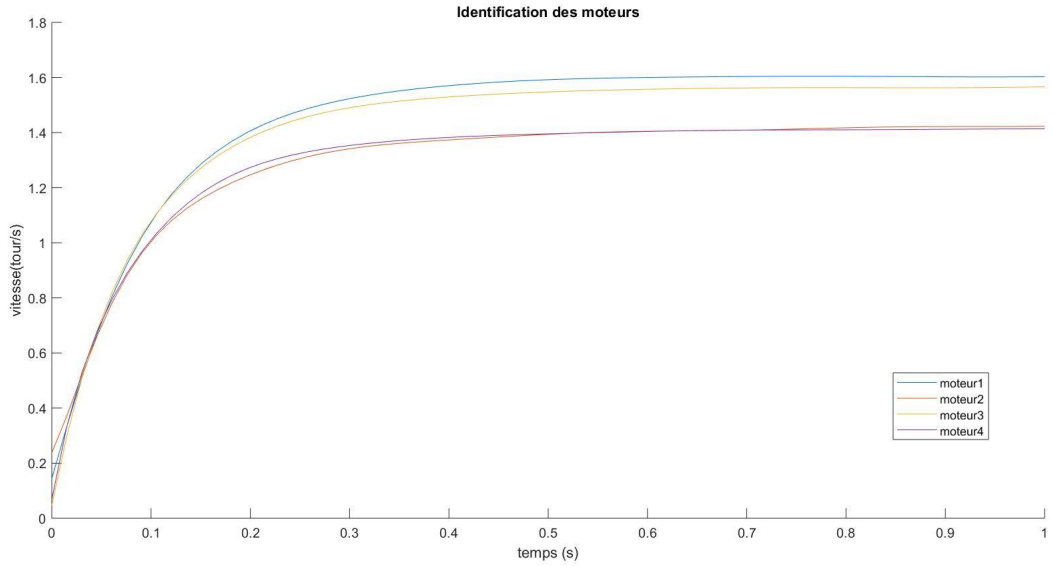


FIGURE 2.8 – Réponse en vitesse des quatre moteurs

2.2.8 Mesure des courants

Diverses méthodes peuvent être considérées pour la mesure des courants des différents systèmes. Le choix dépend de la plage de courant à lire, qui a été déterminée pour les moteurs à partir de leur fiche technique et estimée pour les autres systèmes.

2.3 Planification du développement logiciel

2.3.1 Système de contrôle de versions

Le principal dépôt Git se trouve à l'adresse https://github.com/DODRO35/Design_3.git. Celui-ci est séparé en plusieurs plus petits dépôts afin de plus facilement gérer les différents règles et projets. La liste des identifiants GitHub des membres de l'équipe est présentée dans le tableau 2.2.

nom(s) d'utilisateur	*matricule*	*nom*
douchayoub	111 141 291	Douch, Ayoub
DODRO35	111 221 913	Drolet, Dominic
fjlacroixUlaval	111 130 199	Lacroix, François-Joseph
GASTP33	111 178 727	St-Pierre, Gabriel
gsou & skami	111 182 095	Soulard, Guillaume
nigiroux	111 187 837	Noël-Giroux, Nicolas
aflyingroch	111 181 978	Elfakrani, Rochdi
sophieBRoy	111 188 201	Bourbonnais-Roy, Sophie
VincentM-B	111 183 497	Marois-Boucher, Vincent

TABLEAU 2.2 – Identifiants GitHub des membres de l'équipe

2.3.2 Gestion de projet sur GitHub

Dans ce projet, nous utiliserons grandement les fonctionnalités de gestion de projet de GitHub. La majorité de nos membres sont très familiers avec cette plateforme. Même si la gestion de projet dans GitHub est beaucoup plus axée sur le développement logiciel, il est logique dans notre contexte d'utiliser ces outils puisque nous suivrons une méthode itérative de développement tout au long du projet. En effet, comme la session est séparée en quatre livrables, chaque livrable sera considéré comme une itération à laquelle certaines fonctionnalités doivent être livrées. GitHub est un outil idéal pour notre équipe puisqu'il permet de combiner la gestion de projet et la gestion de versions du code.

2.3.2.1 « Milestones »

Afin de représenter les étapes critiques du développement, nous utiliserons les « milestones » de GitHub. Nous y comptabiliserons toutes les « issues » que nous jugeons nécessaires à la complétion de chaque livrable. Ceci nous permettra donc de visualiser notre avancement dans le projet et de prioriser certaines tâches par rapport à d'autres.

2.3.2.2 « Issues »

Nous utiliserons les « issues » afin de répertorier et découper les différentes tâches en unités de travail plus faciles à gérer et à accomplir. Les issues seront utilisées par tous les

membres de l'équipe, que la tâche soit reliée au développement logiciel ou non. Il est nécessaire que tout le travail à faire soit répertorié en « issues » afin d'avoir une trace de toutes les charges de travail du projet. Un aspect pratique des « issues » est qu'il est possible d'assigner des collaborateurs et de garder la trace de leur évolution à l'aide de commentaires. De plus, nous classifierons ces « issues » selon diverses catégories afin que les membres de l'équipe puissent s'y repérer.

2.3.2.3 Kanban

Nous utiliserons la méthode Kanban afin de garder le fil des différentes tâches sur lesquelles les membres travaillent présentement. Un tableau Kanban est typiquement divisé en trois colonnes, soient : « à faire », « en cours » et « terminé ». Ceci permet de savoir exactement sur quelles tâches les membres de l'équipe travaillent, lesquelles restent à compléter et lesquelles sont terminés. C'est une approche plus visuelle qui incite les membres à faire avancer le projet en s'auto-assignant des tâches. Dès qu'une d'elle est terminée, une autre peut être commencé sans attendre la directive du chef d'équipe. De plus, cela augmente la collaboration puisque les membres peuvent constater si une tâche stagne dans la colonne « en cours ». Dans GitHub, les tableaux Kanban sont appelés des « projects ».

2.3.3 Méthodes de travail

2.3.3.1 Tests

Là où les systèmes sont plus critiques, par exemple au niveau des algorithmes, nous tenterons de travailler en « Test Driven Design » (TDD). Cette technique impose que les tests d'une fonctionnalité soient écrits avant même son implémentation. Le développeur devrait créer un squelette de la fonctionnalité et ensuite écrire les tests. Ces tests représentent en fait les critères d'acceptation de la fonctionnalité. Ainsi, les conditions de succès sont assurées d'être remplies, mais sans plus. Seulement le travail nécessaire est effectué et le niveau de confiance envers la fonctionnalité ainsi que sa robustesse sont largement augmentés.

Idéalement, les tests devraient avoir chacun la plus petite portée possible. Cela est important puisque leurs raisons d'échouer deviennent beaucoup plus claires et identifiables. On sait alors exactement où chercher pour régler le problème. De plus, nous devons nous assurer de tester des comportements et non des résultats précis escomptés. Ainsi, certaines spécificités peuvent changer dans le temps tout en gardant le comportement général acceptable. Par le boulonnage des tests, nous nous assurerons que tout notre système est testé. Le boulonnage des tests consiste à fragmenter les tests du systèmes afin de s'assurer que chacune des composantes fonctionne par elles-mêmes et aussi dans leurs interactions. Il devient alors plus facile de faire évoluer le code dans le temps puisqu'on ne doit pas réécrire une panoplie de tests à chaque fois qu'on fait un changement qui peut affecter plusieurs classes.

Nous tenterons de tester notre système au maximum, mais il faut savoir qu'il est extrêmement difficile de tester des systèmes temps réel et des systèmes qui communiquent sur le réseau.

2.3.3.2 Revues de code

Dans l'optique de produire du code de qualité, nous effectuerons des revues de code courantes. En effet, à chaque fois que quelqu'un veut pousser du code dans la branche de développement, ses changements seront sujets à l'approbation d'au moins un autre membre qualifié de l'équipe. Par exemple, si un étudiant en GLO veut pousser du code dans la branche de développement, il faudra l'approbation d'au moins un autre étudiant en GLO. Ceci nous assurera que tous les membres des sous-équipes sont au courant de l'état du code actuel. Ceci permet aussi de faire appliquer les règles de codage d'une façon plus stricte et donc d'améliorer la qualité du code.

2.3.3.3 Gestion des branches

Les branches serviront à découper les fonctionnalités du robot. Nous calquerons notre approche sur la recommandation GitFlow .

Premièrement, la branche master sera réservée aux versions stables du code que nous considérons viable pour les livrables. Seules deux branches peuvent être « merge » avec master, soient la branche « Hotfix » et la branche « Develop ».

La branche « Hotfix » sert à faire des changements rapides, mais critiques. Ce n'est pas le code le plus élégant, mais c'est du code qui marche et qui résout un problème criant.

La branche « Develop » sert à regrouper toutes les nouvelles fonctionnalités entre les différentes itérations qui sont stables sur la branche « Master ». Lorsqu'une version du code sur cette branche est fiable et stable, elle peut être poussée vers « Master ».

Chaque fonctionnalité du projet est représentée par une branche « Feature » qui démarre à partir d'un point sur la branche « Develop » et qui est poussée sur cette même branche lorsque la fonctionnalité est considérée complète.

Pour chaque « Issue » les membres sont libres de se créer des branches à partir de la branche de fonctionnalité sur laquelle ils travaillent afin de prototyper.

2.3.4 Gestion des fichiers

— Documentation : contient les fichiers nécessaires à la complétion du projet (énoncé, consignes, exemples de rapport, critères, etc.)
— Logiciel : contient tout le code

    - Embedded
    - Systeme
        - BaseStation : Le logiciel qui roulera sur la station de base. Contient la majorité de la logique et la vision. Envoie des commandes au robot.
        - Robot : logiciel monté sur le Raspberry Pi.
- Electrique : contient les circuits imprimés et autres projets reliés aux circuits
- Rapport : contient tout ce qui a trait au Rapport (les images, fichiers reliés, annexes, diagrammes, etc.). Le rapport en tant que tel se trouve sur OverLeaf.

Nous avons décidé de rassembler tout le code tant provenant des étudiants de GIF que les étudiants de GLO dans le même repository sur GitHub. La raison est qu'idéalement, une seule personne devrait pouvoir mettre à jour tout le code et le tester par lui-même en le moins d'étapes possible. De plus, ceci facilite la gestion de projet, car toutes les Issues sont au même endroit.

2.3.5 Langages de programmation utilisés

Comme nous utiliserons un Raspberry Pi comme ordinateur embarqué et que cet ordinateur doit pouvoir traiter des images et communiquer avec la station de base, nous utiliserons le langage Python afin d'exploiter la bibliothèque OpenCV. Pour ce qui est de la station de base, nous devrons aussi traiter des images, planifier des trajectoires et communiquer avec le robot. Le choix du langage Python s'impose ici aussi. Cependant, pour ce qui est de l'interface, nous utiliserons une interface web, puisque nos membres ont de l'expérience dans ce domaine. Nous utiliserons donc JavaScript avec VueJS pour implémenter l'interface.

2.3.6 Règles de codage

Afin de garder notre code propre, compréhensible et maintenable, nous tenterons d'appliquer le mieux possible les conventions présentées dans le livre Clean Code. Nous tenterons d'appliquer les concepts vus dans le cours GLO-4002 Qualité et métriques du logiciel. Ces concepts seront utiles pour nous assurer que notre code soit conçu pour évoluer au fil de la session et empêcher que des gros problèmes surviennent à la fin de la session, au moment le plus critique.

2.3.7 Bibliothèques externes

Pour ce qui est de la vision artificielle, nous utiliserons la bibliothèque OpenCV 3.4.8. Cette bibliothèque sera utilisée tant dans le RaspberryPi que dans la station de base. Puisque la fonctionnalité de planification de trajet est déjà implémentée dans plusieurs bibliothèques, nous choisirons probablement aussi une bibliothèque pour accomplir cette tâche. Nous planifions utiliser des bibliothèques pour les tâches complexes afin de faciliter notre tâche de développement.

Chapitre 3

Design et implémentation

3.1 Avancement de la construction

3.1.1 Pratiques de développement et outils logiciels

Certaines règles et pratiques de développement ont été mises en place afin de standardiser le développement logiciel :

1. L'ensemble du code Python doit respecter la convention PEP8.
2. L'ensemble du code Javascript doit respecter la convention `eslint` standard.
3. Un outil d'intégration continue permet d'exécuter l'ensemble des tests automatiquement.
4. Au moins une revue de code est nécéssaire afin de pouvoir intégrer les changements.

De plus, un wiki régulièrement mis à jour décrit l'ensemble des interfaces, des fonctionnements et des technologies mises en place.

Ces règles et pratiques permettent un meilleur partage des connaissances et des avancements, tout en augmentant drastiquement la qualité du code.

3.1.2 Séquençage des actions

Le système de démarrage de la séquence et du séquençage des actions est pleinement implémenté et fonctionnel. Afin de faciliter l'utilisation, celui-ci se base sur l'ajout de tâches dans une liste. Chaque tâche correspond à une étape critique, comme la lecture d'un panneau

de commande. Chacune de ces tâches comprend le déplacement du robot, qui est délégué à un autre module. Un système de réessaie de tentative permet de réessayer une tâche un certain nombre de fois si jamais cette dernière ne réussit pas. De plus, il est possible de démarrer la séquence à partir d'une tâche particulière et il sera bientôt possible de choisir une tâche singulière. Il est également possible de choisir, au démarrage, la séquence que l'on désire exécuter, ce qui est pratique afin de définir des séquences de tests.

3.1.3 Interface utilisateur

L'interface utilisateur est séparée du système principal de la station de base et communique à l'aide du protocole *« socket »* afin d'obtenir les informations en temps réel. Elle utilise le framework *Vue* afin de séparer les principales parties de l'interface en composantes indépendantes et réutilisables. De plus, un style clair et concis sera mis en oeuvre afin d'en faciliter l'utilisation. Une page additionnelle sera créée pour permettre le contrôle à distance du robot. Cela inclura le déplacement, la rotation, la prise et le dépôt de rondelles et l'allumage de la lumière de la tâche finale.

3.1.4 Communication entre la station de base et le robot

La communication entre la station de base et le robot (RaspberryPi) est également effectuée à l'aide de sockets. Ce mode de communication répond bien au projet, permettant de facilement envoyer des commandes ou faire part de nouvelles informations en tout temps, sans avoir à attendre de réponse. Cela permet de déléguer le retour d'information à d'autres modules du système qui sont indépendants du robot (comme la caméra monde pour déterminer la position du robot). Par contre, certaines tâches ayant besoin de ce retour d'information, par exemple la prise de rondelle, devront attendre la réponse du robot par l'envoie d'un événement de réponse.

3.1.5 Communication entre le RaspberryPi et le STM32

La communication entre le RaspberryPi et le STM32 est faite par protocole UART. Il existe d'autre type de communication que nous aurions pu choisir, comme le SPI ou I2c, mais nous avons choisi ce mode de communication car il est bien adapté dans le cadre du projet. Il est facilement implémentable, peut atteindre des vitesses de communication plus que suffisante et que nous le connaissons bien. En effet, ce protocole ne nécessite que trois connexions, peut atteindre des vitesses de 115200 bps et est supporté autant par le STM32 que le RaspeberryPi. Afin de nous aider à déboguer, nous avons choisis de transmettre les trames sous format *ASCII* et sous le format `INSTRUCT: param1, param2` afin de pouvoir voir clairement les commandes qui sont transmises.

3.1.6 Vision et reconnaissance

3.1.6.1 Caméra monde

Calibration

La caméra monde a été calibrée avec succès. La fonction `calibrateCamera` de `OpenCV` a été utilisée afin d'y arriver. Cette dernière demande la prise de nombreuses photos dans lesquelles un échiquier est présenté sous divers angles et diverses positions. Cela permet au logiciel de déterminer les paramètres intrinsectes de la caméra ainsi que les paramètres de distortion. Afin de valider que la calibration fonctionne, il a été tenté de corriger la distortion de l'image à l'aide des matrices de paramètres préalablement calculées et les résultats était très satisfaisant. Ces matrices sont ensuite sauvegardées en format `JSON`, ce qui permet une analyse humaine et une certaine facilité de déboggage, tout en accélérant le processus de calibration pour les prochaines utilisations. Une configuration différente sera utilisée pour chaque table.

Voici un exemple de test pour la calibration de la caméra à l'aide de la dédistortion de l'image :

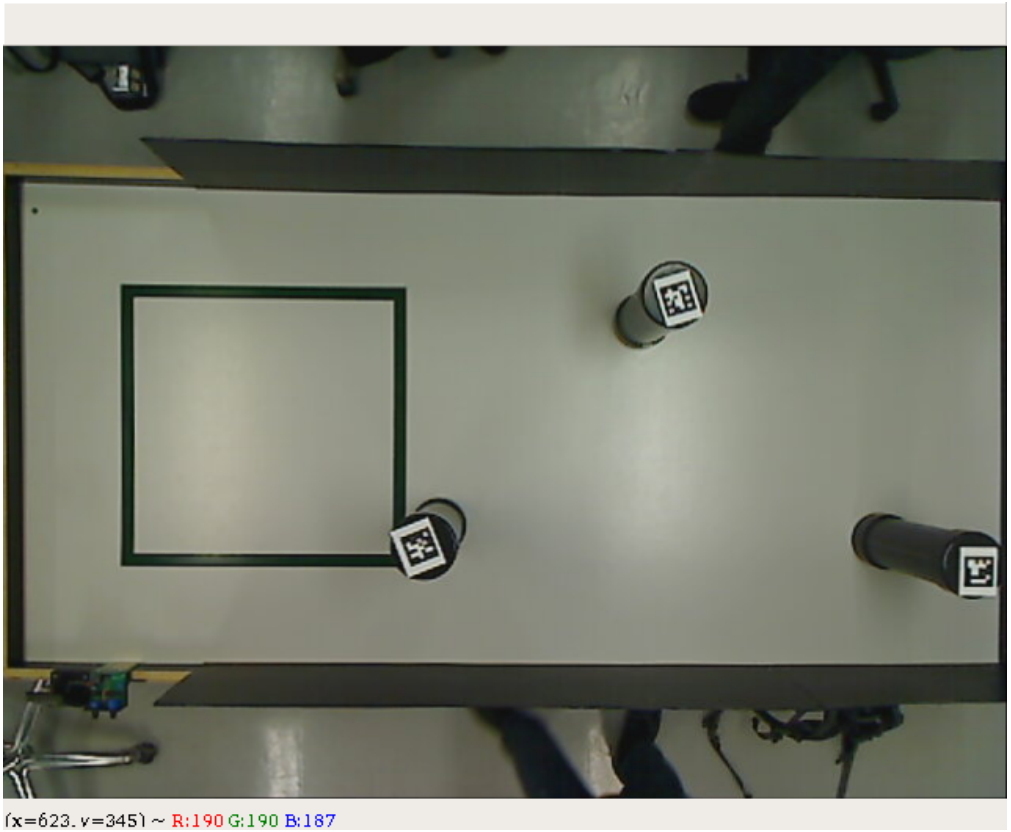


FIGURE 3.1 – Test de la calibration de la caméra

Voici un exemple de mauvaise calibration, clairement visible lors de la dédistortion de l'image :

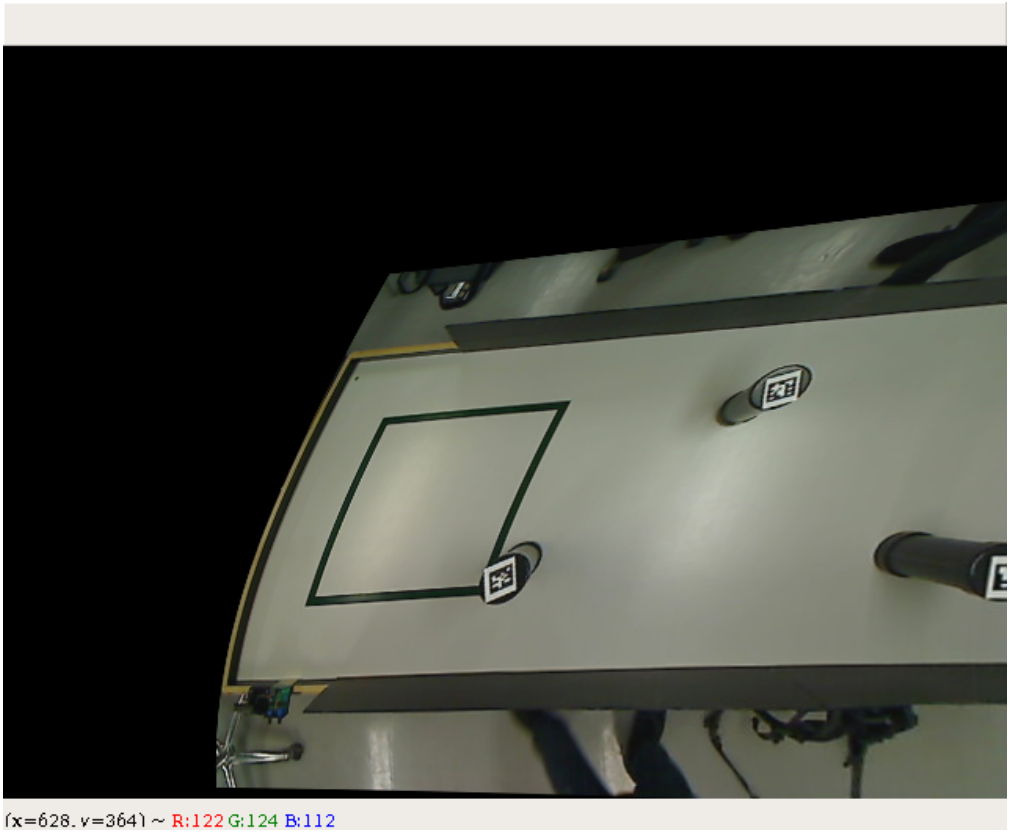


FIGURE 3.2 – Mauvaise calibration de la caméra

Détection des obstacles (piliers)

Au niveau du repérage des obstacles, il a été décidé d'utiliser des marqueurs `Aruco`. Ceux-ci agissent en tant qu'identifiants uniques et sont ainsi facilement dissociables. De plus, il a été remarqué que la librairie `OpenCV` est très efficace pour le repérage de ces marqueurs, et ce, même en situation de faible luminosité ou avec une définition d'image très faible. Par la suite, des fonctions telles que `estimatePoseSingleMarkers` permettent de déterminer les vecteurs de positions et de rotations de ces marqueurs, ce qui permet un repérage tridimensionnel dans le repère monde. Ces dernières utilisent le même principe que celui de la calibration à l'aide de l'échiquier, et comparent les résultats des ratios de mesures avec les données de calibrations. Cela permet d'obtenir une position réaliste et précise.

Voici un exemple de détection des marqueurs `Aruco` ainsi que leur repérage 3D :

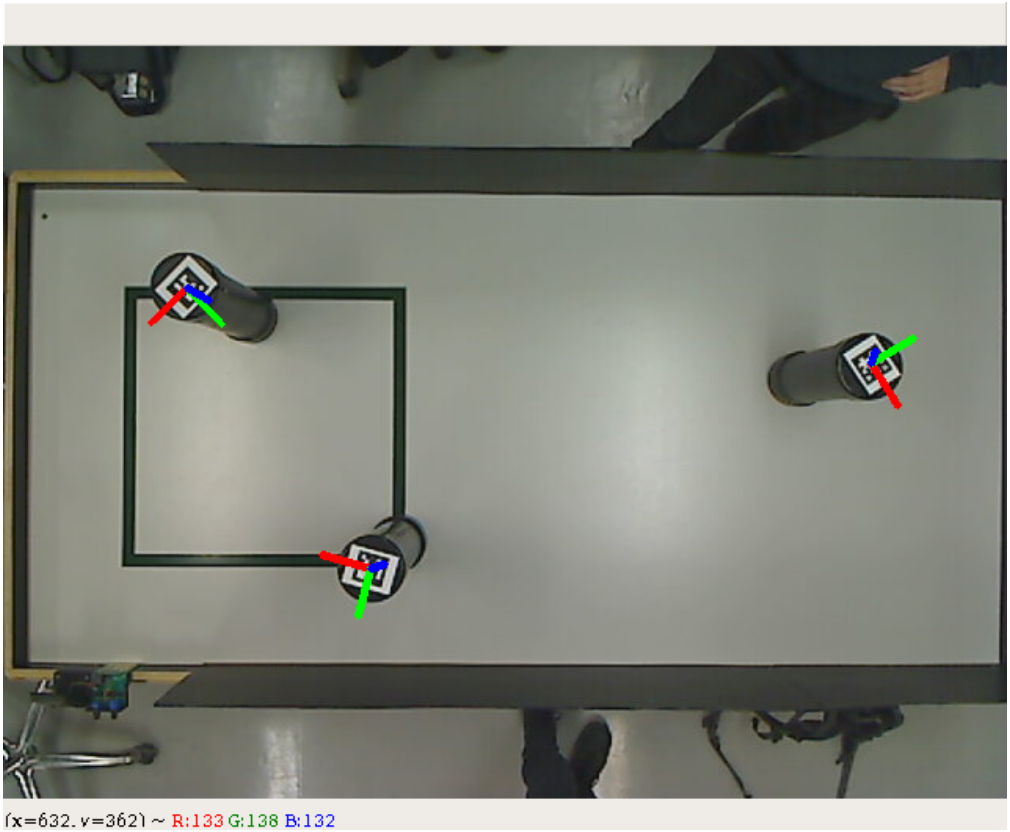


FIGURE 3.3 – Résultat de la détection des marqueurs Aruco

Détection des éléments de la table

Pour la détection des positionnements des coins de chaque table ainsi que ceux de leurs différentes zones ne changeront jamais, il a été décidé, pour le moment, qu'il serait amplement suffisant et précis de déterminer leurs positions manuellement. Une configuration différente sera effectuée pour chaque table.

Détection des rondelles

Pour la détection des rondelles, nous devons manipuler l'image afin de permettre une detection plus fiable. Dans un premier temps, la saturation de l'image est augmentée et une ouverture est appliquée afin de mieux distinguer certaines couleurs telles que le mauve et le blanc. Par la suite, la détection des contours se fait à l'aide de filtres `canny` sur chacune

des couleurs BGR, puis le tout est rassemblé. Cela permet de trouver avec plus de facilité le contour de chacune des rondelles. De cette image de detection des contours, nous pouvons facilement trouver l'ensemble des rondelles en recherchant les cercles dans l'image et en limitant le rayon maximum afin de ne pas détecter les obstacles.

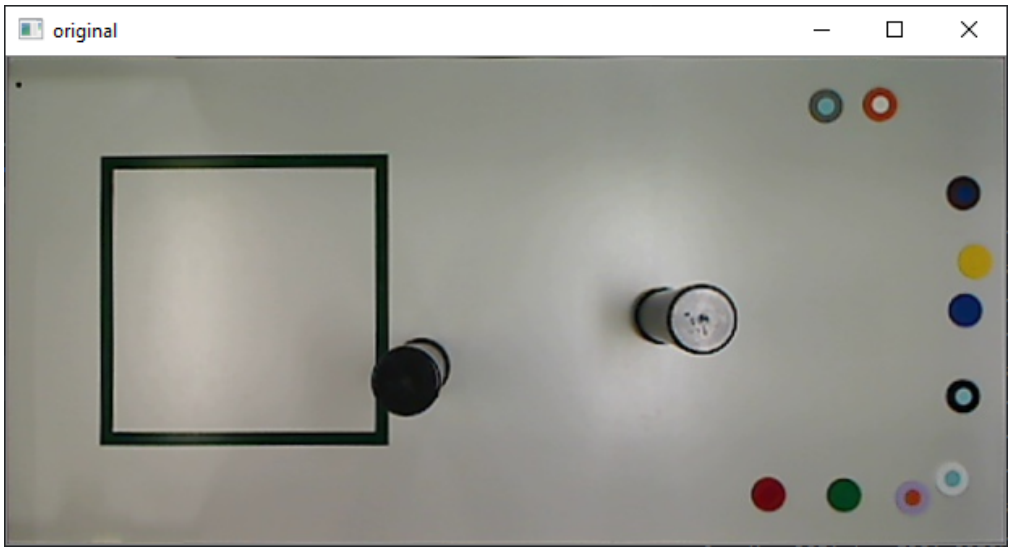


FIGURE 3.4 – Image original

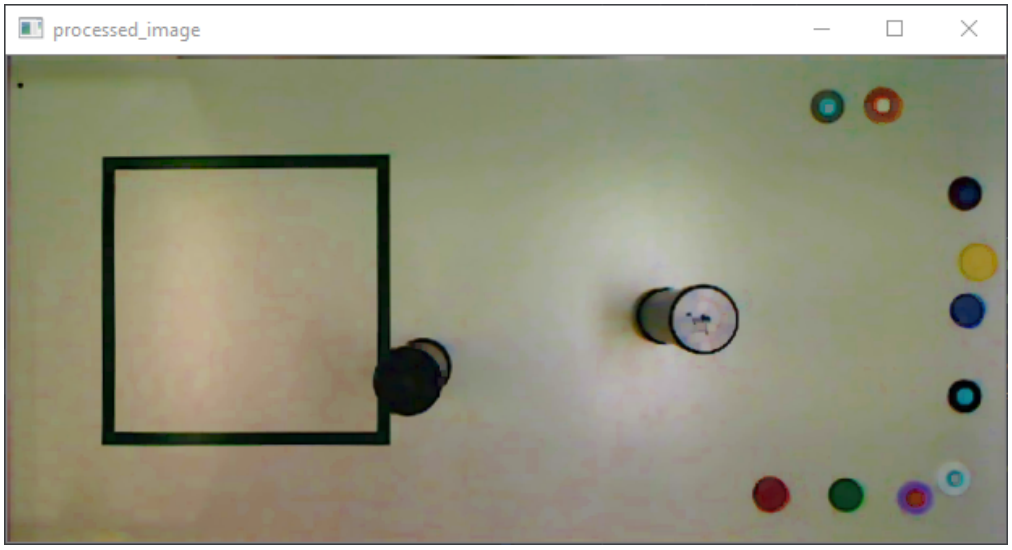


FIGURE 3.5 – Image saturation augmentée et ouverture appliquée

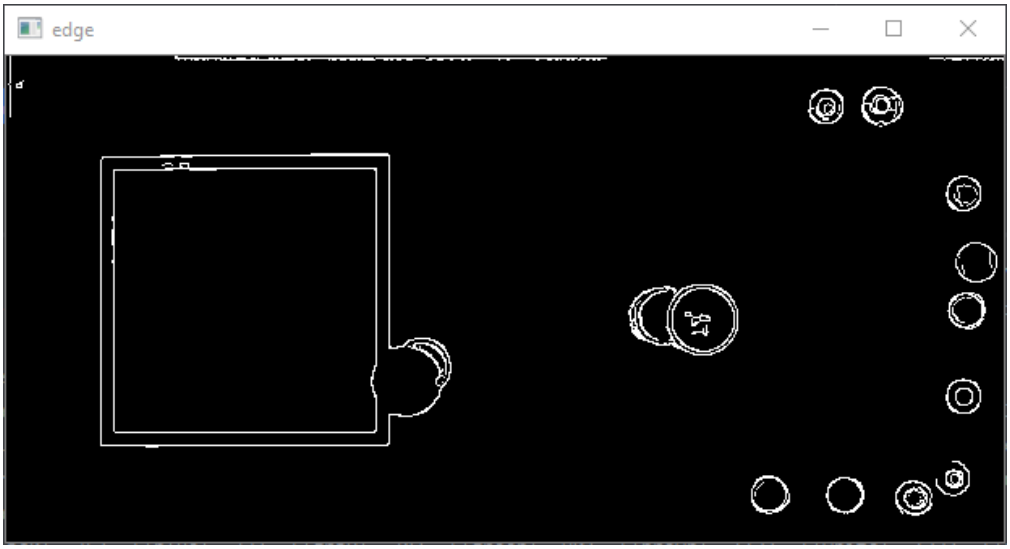


FIGURE 3.6 – Détection des contours

Une fois la position des rondelles trouvées, on peut passer à l'identification des couleurs. L'image est d'abord convertie de BGR vers HSV, ce qui permet de selectionner une plage de couleurs sans se soucier de l'intensité ou du contraste. Par la suite, en itérant sur chaque rondelle, une moyenne de chaque couleur présente est calculée. Celle-ci est alors comparée avec la valeur de chaque couleur déterminée préalablement. Celle correspondant le mieux avec la couleur moyenne est donc la couleur avec laquelle on identifie la rondelle.

Voici le résultat de l'identification des rondelles :

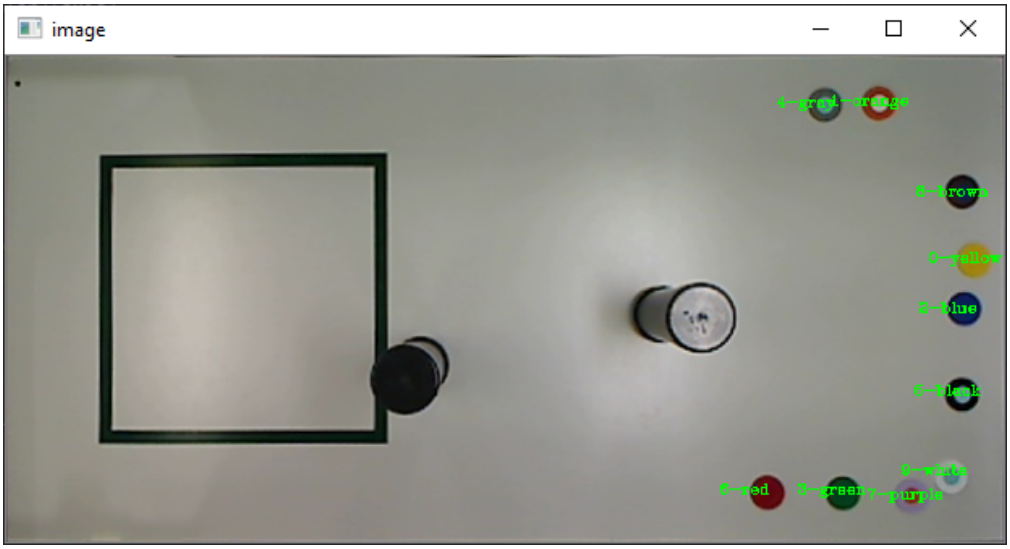


FIGURE 3.7 – Rondelles identifié

3.1.6.2 Caméra robot

La calibration de la caméra robot se fera exactement de la même façon que pour la caméra monde. Bien entendu, une cible de calibration moins volumineuse sera utilisée et une seule configuration sera nécessaire.

3.1.6.3 Détection des tableaux de commandes (flèches)

L'algorithme de détection du tableau de commande des flèches est divisé en deux parties. La première consiste à effacer les informations non pertinentes et potentiellement distrayantes pour l'algorithme. La deuxième étape consiste à isoler le tableau et à détecter la position des flèches dans le tableau.

Suppression d'informations non pertinentes

La première étape consiste à supprimer les portions de l'image qui ne sont pas pertinentes ou même potentiellement nuisibles à la détection des tableaux de commandes. Par exemple, on veut effacer la table de jeux, les rondelles, les murs et tout objets pouvant se trouver au dessus du panneau noir. Bref, nous voulons garder la bande noire qui sert à accrocher les panneaux de commandes et les panneaux de commandes eux-mêmes.

Prenons, par exemple, la figure 3.8 de deux tableaux de commandes rapprochés et de rondelles. Notez que cette image a été prise avec un angle afin de tester la robustesse de l'algorithme.

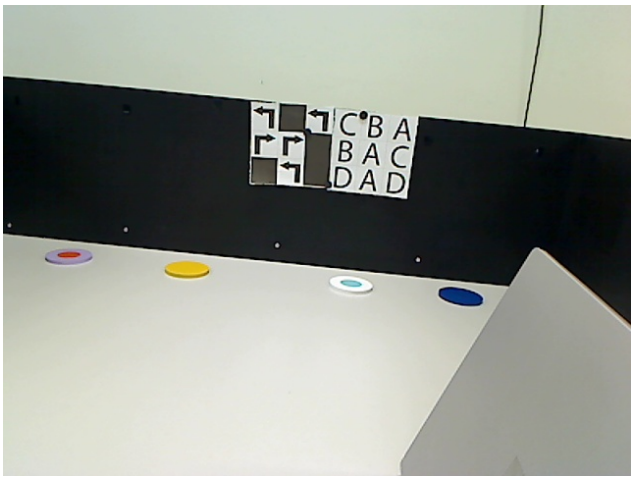


FIGURE 3.8 – Tableaux de commandes vus par la caméra du robot

Afin de supprimer les informations non pertinentes, nous commençons d'abord par tenter

d'éliminer les couleurs qui ne se rapprochent pas du noir ou du blanc. Ainsi, on converti l'image en HSV. Ce format est très utile car il est facile de définir le noir, le blanc et tout le reste des couleurs. Ainsi, on veut sélectionner toutes les couleurs qui ont une saturation et une valeur plus grande qu'un certain niveau. Il a été déterminé expérimentalement que la saturation devrait être plus grande que 40/255 et que la valeur devrait également être plus grande que 40/255. On conserve donc les valeurs dans la tranche du bas et dans le cylindre central de la figure 3.9. La fonction inRange de OpenCV est utilisée pour arriver à cette fin. Le résultat de cette opération peut être vu à la figure 3.10.

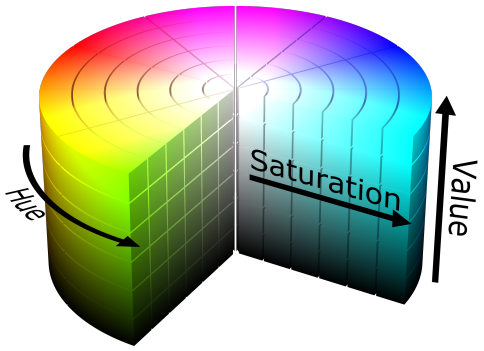


FIGURE 3.9 – Représentation du format de couleurs HSV

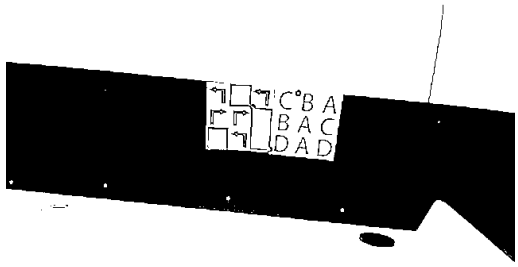


FIGURE 3.10 – Résultat de la fonction inRange

L'étape suivante consiste à fusionner les tableaux de commandes avec l'arrière-plan noir. On utilise donc la fonction d'ouverture. Le noyaux utilisé est de 20x20 pixels. On obtient donc la figure 3.11.

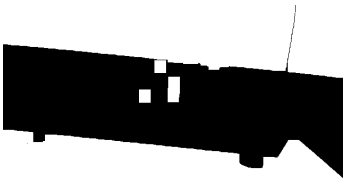


FIGURE 3.11 – Première ouverture



FIGURE 3.12 – Fonction d'érosion



FIGURE 3.13 – Deuxième ouverture

On peut voir que des pochettes blanches ont été créées, mais qu'on commence à remplir les tableaux de commande. Il faut donc effectuer une opération de type « Erode » afin de gruger le blanc de l'image. On utilise donc un noyau de 10x10 pixels et on fait deux itérations pour obtenir la figure 3.12.

Par la suite, on effectue une autre opération d'ouverture afin de remplir complètement les derniers trous blancs et afin d'élargir l'arrière-plan noir. Ceci est utile car les opérations précédentes peuvent avoir rétréci la zone noire, ce qui ferait en sorte qu'on ne sélectionnerait pas tout les tableaux de commande. On utilise encore une fois un noyau de 20x20 pixels et on obtient la figure 3.13.

Ensuite, on inverse l'image et on effectue une opération de « et » logique entre l'image originale et l'inversion. On obtiens alors la figure 3.14.

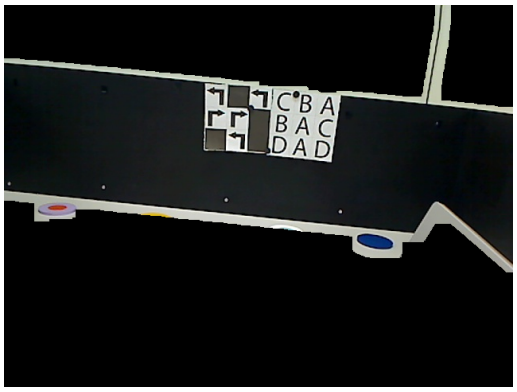


FIGURE 3.14 – Résultat de l'addition du masque et de l'image originale.

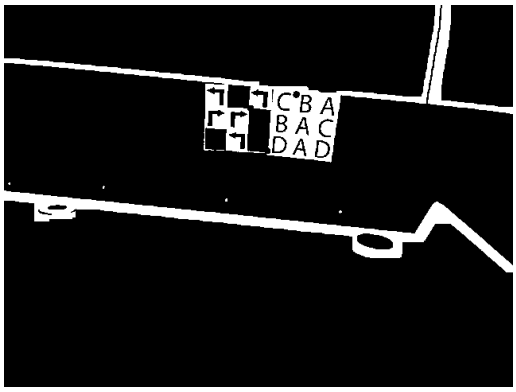


FIGURE 3.15 – Panneaux en noir et blanc

Détection du panneau de flèches

Pour la détection du panneau des flèches, on commence par convertir l'image en tons de gris. Ensuite, on converti cette image en noir et blanc. Les valeurs de gris supérieures à 127 seront converties à 255 et les valeurs plus petites ou égales à 127 seront converties en 0. On obtient donc l'image figure 3.15.

On tente donc de retrouver dans l'image où se trouve le panneau complet des flèches. On procédera donc à la recherche du patron des flèches horaire et anti-horaire dans toutes les positions possibles de l'image.

Étant donné qu'on ne sait pas préalablement à quelle distance on se trouve des panneaux et l'angle dans lequel ils se trouvent par rapport à la caméra, il faudra explorer plusieurs mises à l'échelle des patrons et plusieurs orientations de ceux-ci. On utilisera une méthode itérative pour déterminer les meilleurs paramètres.

On aura donc recours à la méthode `matchTemplate` de OpenCV, qui parcours toute l'image et qui estime la corrélation entre le patron et toutes les sections de l'image. On utilisera cette méthode avec les deux patrons de flèches disponibles. Une corrélation de 1 voudrait dire que la portion de l'image correspond exactement au patron tandis qu'une corrélation de 0 voudrait dire qu'il n'y a aucun lien entre le patron et la section analysée.

On obtient donc une liste de corrélations selon la position dans l'image. On peut visualiser cette liste et on obtient la figure 3.16.

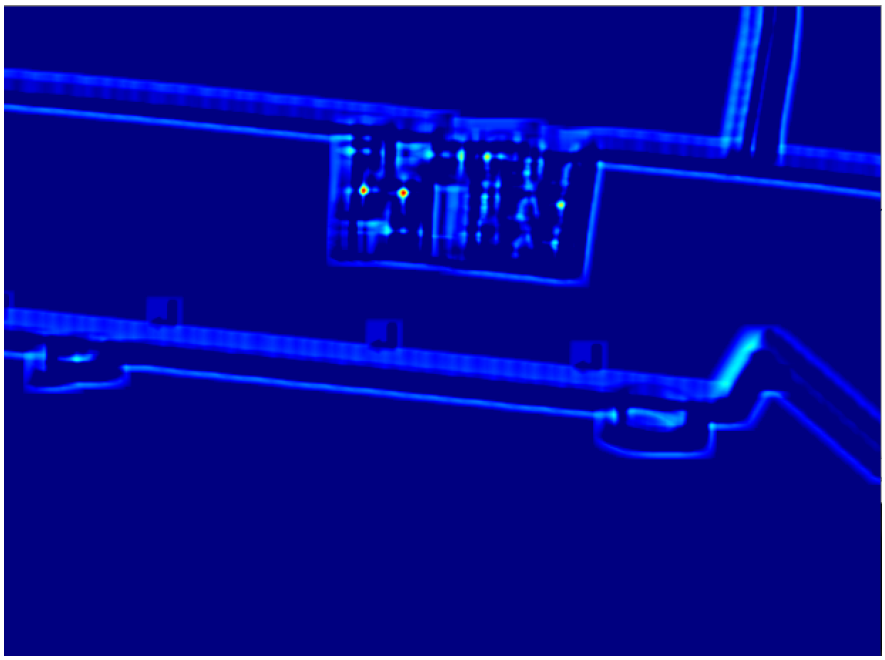


FIGURE 3.16 – Distribution de la corrélation avec la flèche du sens horaire et l'image de base.

Ici, ce sont les flèches du sens horaire qui ont été estimées. On peut bien voir qu'il y a deux zones très corrélées au centre du panneau de gauche. On peut aussi constater que la nature géométrique des lettres du panneau de droite produit des faux positifs. En jouant sur les paramètres de la méthode itérative, on peu arriver à réduire ces faux positifs en demandant à ce que l'algorithme continue à itérer tant qu'il ne trouve pas des corrélations très grandes, par exemple 90

Les données de corrélation extraites servent ensuite à localiser le tableau dans l'image. On commence par trouver le point ayant la corrélation la plus élevée pour retrouver les autres flèches. Puisqu'on ne peut pas savoir d'avance quelle flèche produira la plus grande corrélation, il faut assumer que la flèche avec la plus haute corrélation peut se trouver n'importe où dans le tableau. Il faut donc former une griller de 5 cases par 5 cases chacune de la grosseur de la case de la meilleure flèche. Il faut donc aller estimer les valeurs de corrélations dans les 24 cases adjacentes afin de déterminer quelle section de 3x3 cases donne la corrélation la plus élevée.

Pour ce faire, on sélectionne la portion de l'image qui est centrée sur la meilleure corrélation et qui englobe une région de 5x5 cases équivalentes. Avec cette image, on peut procéder à l'analyse des valeurs de corrélation des autres flèches. On va donc chercher la valeur la plus élevée dans l'entourage de chacune des cases. Effectivement, il y a beaucoup d'incertitude sur la position des cases alors on ne peut se contenter d'aller évaluer un seul point. On garde la valeur la plus élevée entre la corrélation horaire et anti-horaire. Si le maximum de ces valeurs ne dépasse pas 0.45, on ignore et on considère qu'il s'agit d'un des carrés noirs ou d'une section extérieure au tableau des flèches. Ces carrés se voient attribuer la note 0.

Basé sur ces estimations, on peut estimer quelle grille 3x3 contenue dans la grille 5x5 obtient le meilleur score, qui correspond à la somme des meilleures corrélations trouvées pour chaque case. On obtient donc une prédiction des flèches présentes dans le tableau comme on peut le voir dans l'exécution du code à la figure 3.17. On a une liste qui représente le panneau de gauche à droite et de haut en bas. On a donc pour la première rangée [2,0,2] qui signifie [anti-horaire, noir, anti-horaire] et ainsi de suite pour les deux autres rangées.

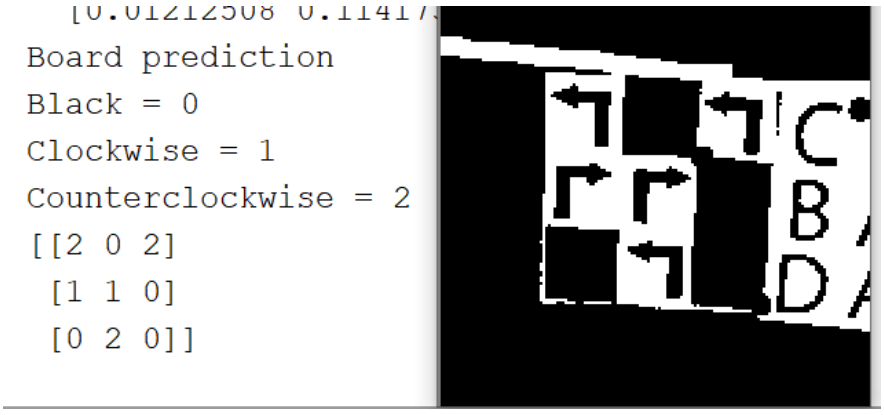


FIGURE 3.17 – Prédiction du positionnement des flèches dans le tableau de commandes.

À venir

Pour le moment, l'approche semble être fiable, mais plus de tests sont nécessaires pour savoir exactement dans quels cas la méthode échoue. Il faudra implémenter un mécanisme d'exceptions afin de signaler au robot que le tableau n'a pas été trouvé ou bien que les valeurs semblent erronées. Nous avons déjà implémenté le concept de « retryable » ce qui veut dire qu'une tâche qui échoue pourra se réinitialiser avec des paramètres différents afin d'obtenir un résultat plus fiable.

Dans l'implémentation de l'approche de détection des flèches, il a été remarqué que des images floues peuvent produire des résultats erronés. Il faudra donc détecter ces mauvaises images pour les rejeter et recommencer la tâche de détection après avoir déplacé le robot pour obtenir une meilleure image.

Afin d'avoir une bonne compréhension du système de détection et de ses faiblesses, il faudra mettre un place un processus de tests comme il a déjà été fait pour la détection des rondelles. Ceci est extrêmement utile lorsque des nouveaux changements sont faits car on peut tester notre algorithme sur des centaines d'images et ainsi avoir confiance que l'algorithme est robuste. Des images testant les limites de l'algorithme ont déjà été capturées, mais il reste à automatiser le processus de tests.

L'algorithme de détection du tableau de lettres n'a pas encore été implémenté, mais il devrait être extrêmement similaire à celui de la détection des flèches. Il sera probablement plus facile à implémenter puisque les tableaux de lettres seront toujours remplis et n'auront pas de sections noires. Il sera donc encore plus facile de déterminer la région complète du tableau.

3.1.7 Planification de trajectoire

L'algorithme de planification de trajectoire n'a pas encore été implémenté. Cependant, nous avons déjà décidé que nous utiliserons l'algorithme A* pour déterminer la trajectoire optimale, puisque c'est l'algorithme de pathfinding le plus rapide et qu'il n'est pas nécessaire d'utiliser une méthode déterministe. De plus, cet algorithme sera appliqué sur une grille de positions 2D (avec un degré de précision prédéterminé). Sur cette grille sera présentée les positions des obstacles, en incluant leur diamètre ainsi qu'un certain espacement. Cela permettra d'ajouter une confiance supplémentaire afin de s'assurer qu'aucun obstacle ne sera heurté durant la séquence.

3.1.8 Alimentation

3.1.8.1 Description des composantes

L'alimentation a le rôle de fournir l'énergie électrique aux autres systèmes. Elle comprend la batterie et les différents régulateurs qui créent les différents niveaux de tension requis. Dans notre cas, les systèmes nécessitent une alimentation de 5V ou 12V.

3.1.8.2 Solutions retenues et considérées

Pour le choix de la batterie, une batterie acide-plomb a été choisie. Les avantages de cette chimie sont la grande capacité, un prix assez faible pour cette grande capacité et une tension très stable sur la plage d'utilisation. Aussi, la tension de 12V (12-13.5) d'une telle batterie permet l'utilisation directe sur le pont en H des moteurs, un régulateur 12V n'est donc pas nécessaire. Ensuite, une batterie au plomb ne nécessite pas de BMS selon les contraintes du projet.

Le poids plus grand comparé à une batterie au lithium ne cause pas de problème. Historiquement, plusieurs équipes ont choisi d'ajouter du poids à leur véhicule pour augmenter l'adhérence. Le poids de la batterie servira donc à atteindre cet objectif.

3.1.8.3 Moyens techniques

Une fois la chimie choisie, la capacité désirée a été déterminée en estimant la consommation du robot. Le Raspberry Pi et l'électronique de support consomme au maximum 5A sur le 5V. Quant aux moteurs, le fusible de 3A limite le courant maximal. Ainsi, on détermine une puissance maximale possible de

$$5V \cdot 5A + 12V \cdot 3A \approx 60W$$

La batterie choisie (APC-114) était déjà en possession de l'équipe et a une capacité de 6Ah. Outre l'avantage du prix sauvé en réutilisant la batterie, une si grande capacité permet de faire plus d'une heure d'essais. La capacité à été validée en effectuant de longs essais, puis en mesurant la capacité restante de la batterie.

Aussi, la capacité d'une batterie au plomb peut facilement être estimée par la lecture de sa tension. Une lecture de tension lors des moments de basse consommation est donc suffisante pour obtenir la charge de la batterie.

L'alimentation des systèmes électronique de 5V est créée à partir d'un hacheur dévolteur. Cette solution permet une grande efficacité de conversion. Le système actuel utilise un régulateur commercial, qui acheté avant que ça ne devienne interdit d'en utiliser un. Ce régulateur, permettant de passer de 12V à 5V avec 5A en sortie, sera utilisé jusqu'à la conception de la deuxième version du PCB.

3.1.9 Préhenseur

3.1.9.1 Description des composantes

Le préhenseur est composé du support à rondelles et du système de levage.

3.1.9.2 Solutions retenues et considérées

Pour le système de levage, la simplicité et la fiabilité ont été les critères du design. Ainsi, il est composé d'un seul axe rotatif, créé par un servomoteur. L'utilisation d'un servomoteur permet un contrôle en position sans effort. Le mouvement de rotation est converti en translation par un système pignon et crémaillère. L'effort du servomoteur est ainsi directement proportionnel à la gravité et donc la masse de l'objet soulevé. La légereté du système permet de facilement distinguer entre une ou deux rondelles soulevées en mesurant le courant du servomoteur.

Le système de levage ajoute des contraintes sur le support de rondelles. Celui-ci doit pouvoir entrer et sortir de l'axe du système de levage. Pour cela, deux solutions ont été envisagées. Premièrement, un deuxième axe de translation peut faire sortir et entre un panier dansla zone sous le préhenseur. Toutefois, l'utilisation d'un autre axe de rotation permet aussi bien d'atteindre cet objectifs et c'est la solution qui a été retenue pour sa simplicité.

3.1.9.3 Moyens techniques

Finalement, la solution qui a été retenue pour saisir les rondelles est un électroaimant 5kg, 12V, 0.35A. Les paramètres sont plutôt arbitraires et correspondent à ce qui était disponible à un bas coût. En effet, il est facile de faire varier la force de l'aimant par réduction de la tension d'alimentation et par l'augmentation de la distance minimal entre l'aimant et la pièce (en ajoutant un séparateur). La tension et la distance idéale ont été déterminées par essais-erreur pour fournir la force minimale à la levée d'une seule rondelle.

Le circuit de contrôle du préhenseur (figure 3.18) a été concu de façon à être le plus simple possible tout en permettant un certain contrôle de la tension aux bornes de l'aimant. Le circuit le plus simple et le moins couteux pour cela est un simple MOSFET activé par un signal PWM provenant des systèmes embarqués. Le MOSFET a peu de critères de sélection à cause du bas courant (300mA) et de tension (12V) qu'il doit supporter. Le DMG2302UK-7 a été choisi pour son faible coût (0.46$). La validité de ce circuit est facilement vérifiée en l'essayant avec l'électroaimant.

3.1.10 Mesure de resistance

3.1.10.1 Description des composantes

Le système de mesure de résistance inclut le circuit de lecture de résistance et le système de désactivation de l'ohmmètre.

3.1.10.2 Solutions retenues et considérées

La désactivation de l'ohmmètre est assez simple. Il suffit de connecter ensemble les broches de ground et de relais.

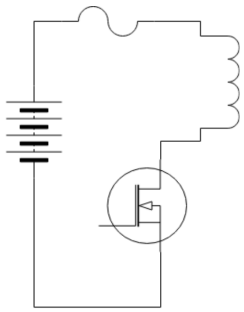


FIGURE 3.18 – Circuit de contrôle de l'électroaimant du prehenseur

Pour la lecture de résistance, deux possibilités ont été considérées. Premièrement, il est possible d'utiliser la loi d'Ohm pour déterminer la résistance. Ainsi, un courant précis est injecté dans la résistance. Il est ensuite possible de mesurer la tension avec l'ADC du STM32, qui a une précision de 12bits, ou bien $806\mu V$. Il n'est pas possible de mesurer l'entièreté de la plage des résistances avec cette précision, alors un système permettant de choisir parmi 4 valeurs de courant a été construit (figure 3.19). Ceci permet d'obtenir un intervalle de tension entre les mesures beaucoup plus raisonnable. Toutefois, lorsque le circuit a été réalisé, la précison était insuffisante. Une autre méthode a donc dû être envisagée.

Si l'ADC est limité à 12bits, une mesure de temps avec l'horloge 168MHz du STM32 est beaucoup plus précise. En effet, cela permet d'avoir une résolution d'environ 6ns avec une variation de quelques dizaines de ppm.

3.1.10.3 Moyens techniques

Ainsi, un circuit basé sur la mesure de temps à été développé (figure 3.20). Ce circuit est basé sur un délai entre l'événement d'entrée et la sortie du comparateur causé par le circuit RC. Ce délai varie en fonction de la résistance. Le délai peut initialement être estimé :

$$V_c = 1 - e^{-\frac{t}{RC}}$$

Sachant que le temps mesuré est quand V_c atteint la moitié de la tension d'alimentation,

$$\begin{aligned} 0.5 = 1 - e^{-\frac{t}{RC}} \\ \implies \ln 0.5 = -\frac{t}{RC} \\ \implies R = \frac{t \ln 2}{C} \end{aligned}$$

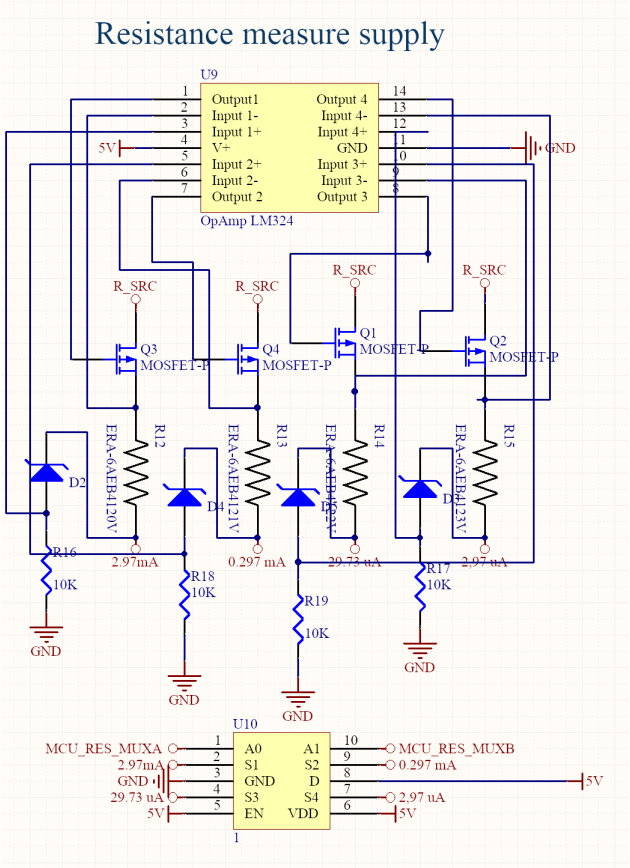


FIGURE 3.19 – Circuit de lecture de résistance avec lecture de tension

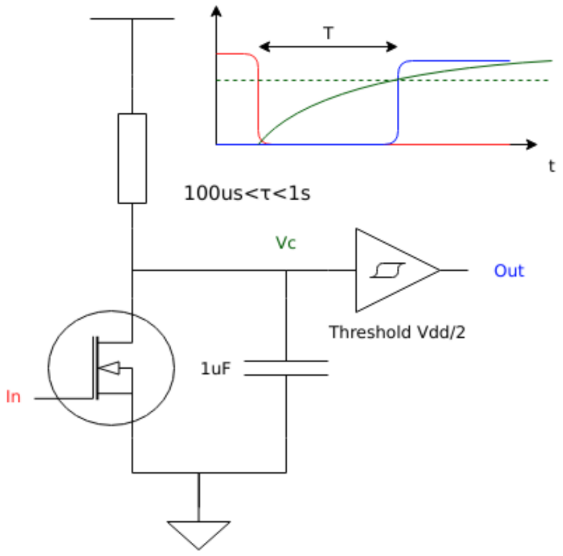


FIGURE 3.20 – Circuit de lecture de résistance avec mesure de temps

Avec cette formule il est possible d'atteindre 3 chiffres significatifs sur la résistance. Une présicion plus élevée est obtenue en calibrant le système avec des valeurs connues.

Valeur affiché sur multimètre (Ω)	Valeur mesurée (Ω)
107.3	106.6
196.2	196.9
489	488
501	502
991	990
1003	1001
1111	1106
12 000	11 990
13 000	12 888
40 000	39 980
125 000	125 100
920 000	920 101
930 000	930 243

TABLEAU 3.1 – Mesure de différentes valeurs sur toute la plage à l'aide du circuit RC

Le tableau 3.1 permet de confirmer que le circuit RC fournit la précision nécessaire sur toute la plage pour effectuer la mesure de la résistance.

3.1.11 Aquisition

3.1.11.1 Description des composantes

Le systèmes d'aquisition inclue les différents capteurs de courant, la circuitrie qui leur est attachée et le système qui numérise la valeur de courant.

3.1.11.2 Solutions retenues et considérées

Afin de simplifier le design, des solutions intégrées ont été recherchées. En effet, il existe plusieurs circuits imprimés contenant une sonde de courant et un amplificateur fournissant une tension pouvant être directement lue par l'ADC du STM32.

3.1.11.3 Moyens techniques

Le ACS70331 rencontre tous ces critères. Son bas coût (2.5$) en fait également une bonne solution. Il suffit de passer le courant dans ce circuit imprimé pour avoir une tension proportionnelle à la sortie. Cette tension peut alors être lue par un ADC du STM32.

3.1.12 Traction

3.1.12.1 Description des composantes

Le système de traction inclut les moteurs, leurs encodeurs, le pont en H, qui sont tous fournis, mais aussi le système de contrôle, inclut dans le logiciel embarqué.

3.1.12.2 Solutions retenues et considérées

Le système embarqué choisi, une carte de développement pour STM32, supporte le logiciel d'asservissement et de contrôle des moteurs. Il y a plusieurs façons de propager la commande de mouvement déterminée par les systèmes informatiques vers les roues. La solution choisie prend en entrée deux vitesses pour les deux axes du robot. Ainsi, le mouvement diagonal est possible. À cela s'ajoute une vitesse de rotation désirée. Ces trois variables sont simplement converties en vitesse désirée pour chaque roue en additionnant les effets de ces trois commandes. Ensuite, un contrôle par régulateur PI est réalisé afin d'atteindre la vitesse désirée sur chacune des roues. La figure 3.21 présente un schéma de contrôleur.

3.1.12.3 Moyens techniques

Le signal de l'encodeur, ainsi que la commande de vitesse, sont filtrées avec un filtre numérique simple :

```
filtered = (1 - strength) * filtered + strength * value
```

Ce filtre est équivalent à la fonction de transfert suivante :

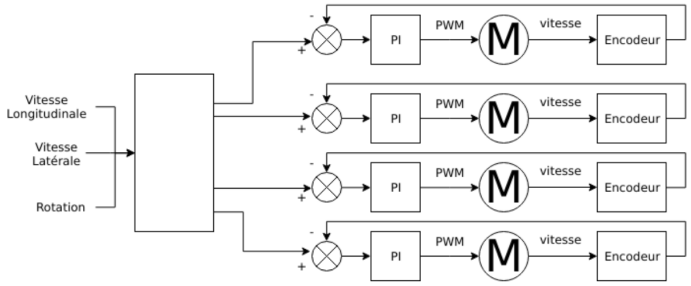


FIGURE 3.21 – Schéma de l'algorithme de contrôle moteur

$$
\begin{aligned}
& y(k+1) = (1-C)y(k) + Cx(k) \\
\implies & zy(z) = (1-C)y(z) + Cx(z) \\
\implies & (z + C - 1)\, y(z) = Cx(z) \\
\implies & \frac{y(z)}{x(z)} = \frac{C}{z + C - 1} \\
\implies & G(z) = \frac{Cz^{-1}}{1 - (1-C)z^{-1}} \\
\implies & G(s) = \frac{1}{1 + \tau s} \wedge e^{-T/\tau} = 1 - C
\end{aligned}
$$

La période choisie est de $T = 0.01$. Ceci permet d'avoir une fréquence assez rapide pour le contrôle (100Hz), tout en laissant amplement de puissance de calcul pour les autres systèmes.

Le choix de la constance de temps est effectué en fonction du comportement des moteurs en vitesse. La fréquence de coupure choisie est de 20Hz. Il est finalement facile de trouver le paramètre des filtres numériques :

$$1 - C = e^{-\frac{T}{\tau}} \implies C = 1 - e^{-\frac{1/100}{1/20}} \approx 0.18$$

3.2 Tests unitaires

3.2.1 Résultats des tests

```
$ pipenv coverage run
.........
```

```
Ran 9 tests in 0.395s

OK
```

3.2.2 Résultats de couverture des tests

```
$ pipenv coverage run && pipenv coverage report
```

Name	Stmts	Miss	Cover
src/domain/GeneralEventEmitter.py	5	5	0%
src/domain/GlobalInfos.py	16	8	50%
src/domain/Observer.py	10	10	0%
src/domain/ThreadManager.py	8	2	75%
src/domain/enums/ResistorColors.py	12	12	0%
src/domain/robot/RobotEventEmitter.py	22	22	0%
src/domain/robot/RobotInfos.py	100	100	0%
src/domain/sequence/Retryable.py	15	0	100%
src/domain/sequence/SequenceEventEmitter.py	14	4	71%
src/domain/sequence/SequenceRunner.py	21	21	0%
src/domain/sequence/Task.py	10	2	80%
src/domain/sequence/TaskContainer.py	39	6	85%
src/domain/sequence/TaskContainerFactory.py	13	13	0%
src/domain/sequence/exceptions/RetriesExceededException.py	3	0	100%
src/domain/sequence/exceptions/RetryException.py	5	1	80%
src/domain/sequence/exceptions/WarningException.py	3	1	67%
src/domain/sequence/task_containers/MainTaskContainer.py	8	8	0%
src/domain/sequence/tasks/RobotMoveLoopTask.py	21	21	0%
src/domain/vision/ImageProcessing.py	43	7	84%
src/domain/vision/PuckFinder.py	110	18	84%
src/domain/vision/TableCropper.py	51	19	63%
src/domain/vision/aruco/ArucoFinder.py	27	27	0%
src/domain/vision/calibration/CameraCalibrationRepository.py	12	12	0%
src/domain/vision/calibration/CameraCalibrator.py	38	38	0%
src/domain/vision/calibration/ImageUndistorder.py	12	12	0%
src/domain/vision/panels/ArrowBoardDetector.py	206	206	0%

src/domain/vision/panels/BoardSegmentation.py	22	22	0%
src/infrastructure/event_emitters/socket/GeneralSocketEventEmitter.py	9	9	0%
src/infrastructure/event_emitters/socket/RobotSocketEventEmitter.py	23	23	0%
src/infrastructure/event_emitters/socket/SequenceSocketEventEmitter.py	16	16	0%
src/infrastructure/thread_managers/SocketThreadManager.py	11	11	0%
src/interfaces/socket_api/DebugRobotSocketEventHandler.py	12	12	0%
src/interfaces/socket_api/RobotSocketEventHandler.py	23	23	0%
src/interfaces/socket_api/SequenceSocketEventHandler.py	18	18	0%
TOTAL	958	709	26%

Chapitre 4

Rétrospectives

4.1 Gestion du contenu

4.1.1 Progrès

Après la démonstration du livrable 2, certaines avancées majeures ont été effectuées. Ces dernières comprennent la détection de la base des obstacles, le pathfinding et l'interface.

4.1.1.1 Calibration de la caméra monde et détection des obstacles

La détection des obstacles a été complétée et largement améliorée en 3 parties.

a. Amélioration de la calibration

La calibration étant maintenant fonctionnelle, un autre problème s'est ajouté à la liste. Tout d'abord, les images de calibration ayant été prises à une résolution de 960x720 pixels, la précision de la détection n'était pas entièrement optimale. Des photos à pleine résolution (1600x1200 pixels) s'en sont suivies, résultant en une meilleure précision.

Deuxièmement, un bogue s'est introduit lorsqu'est venu le temps de prendre des captures en temps réel avec la caméra monde. La capture vidéo à pleine résolution étant particulièrement lente, il était préférable de revenir à une résolution d'environ 960x720 pixels. Par contre, il a été observé qu'un changement de résolution ou de ratio d'aspect affecte la matrice de calibration. Heureusement, une méthode simple consiste à simplement multiplier f_x et f_y par les ratios de changements en largeur et hauteur de l'image capturée par rapport aux images de calibration. Les dimensions des images de calibration sont ainsi également sauvegardées afin de pouvoir utiliser la résolution désirée sans aucune contrainte.

b. Amélioration de la détection des marqueurs

La détection de la position 2D (position 3D projetée sur le plan image) des marqueurs `Aruco` fonctionnait déjà très bien. Par contre, un des problèmes observés fût le manque de stabilité de la détection de l'orientation du marqueur. Les axes affichés changeaient constamment d'orientation et de sens, ce qui empêchait une future détection de la position à la base des obstacles. Il a été observé qu'un des principaux problème était que l'algorithme de détection des marqueurs avait de la difficulté à bien identifier les coins noirs et détectait parfois les bordures noires de l'obstacle à la place. Cela a été corrigé à l'aide de l'impression de nouveaux marqueurs couvrant l'ensemble de la surface supérieure des obstacles, en gardant une bordure blanche d'au moins 2cm autour du marqueur. La stabilité de la détection de l'orientation 3D des marqueurs a ainsi drastiquement été améliorée.

c. Simplification et stabilisation

Par contre, le problème de stabilité semblait tout de même persister, plus particulièrement lorsque les marqueurs se situaient près de la caméra (dans les axes X et Y). L'algorithme de OpenCV semble avoir des problèmes de précision lorqu'un des angles devient trop petit (ici l'angle γ se rapportant à l'axe Z). Après réflexion, il a été premièrement remarqué que les obstacles sont toujours parallèles à l'axe Z, et donc l'angle γ (celui qui semble faire défaut lors de la détection de l'orientation) pouvait simplement être omis et forcé à 0. Deuxièmement, il n'est pas non plus utile de déterminer les angles α et β (par rapport aux axes X et Y, respectivements) puisque ceux-ci ne serviront tout simplement à rien. Nous avons donc forcé tous les angles relatifs aux marqueurs `Aruco` à 0, ce qui force la détection à prendre les angles qui sont directement calculés depuis la matrice de la caméra. Le résultat est maintenant extrêmement stable, peu importe la position de l'obstacle.

Finalement, la détection de la position au sol des obstacles fût très facile et rapide à implémenter. Ayant maintenant un système d'axe fiable, il suffisait d'indiquer une position relative en Z correspondant à la hauteur d'un obstacle, ce qui a pour effet de « descendre » la position en suivant l'axe Z. Nous pouvons observer dans la figure 4.1 les résultats de la détection des coins des marqueurs, de leur système d'axe ainsi que de la position au sol des obstacles. On peut également y observer les nouveaux marqueurs `Aruco` utilisés. Notons que l'axe Z est inversé puisqu'il correspond à celui de la caméra.

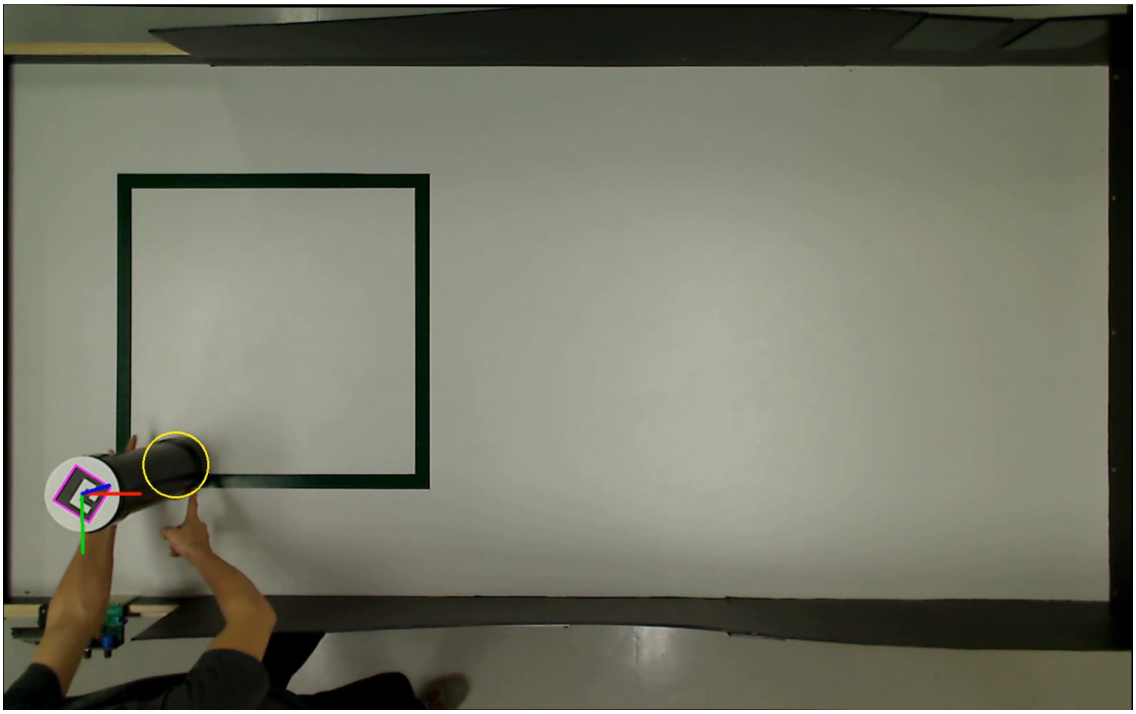


FIGURE 4.1 – Détection améliorée de la base des piliers

4.1.1.2 Pathfinding et systèmes de coordonnées

Le pathfinding n'étant pas présent lors du livrable 2, une première version a été développée dans les semaines qui ont suivies. Cette dernière est constituée de 2 parties principales. De plus, un système de changements de coordonnées a dû être mis en place afin de grandement simplifier et généraliser l'algorithme.

a. Création de la grille de recherche

Tout d'abord, afin de pouvoir lancer un algorithme de recherche, il fallait créer une grille de noeuds sur laquelle la recherche pourra s'effectuer. Chaque noeud de cette grille contient une certaine valeur indiquant soit la présence d'un obstacle (indiqué par le nombre 0), soit le poids du chemin allant vers ce noeud, et représente une position 2D dans le repère de la table ($(0, 0)$ étant le coin inférieur gauche). La grille respecte également une certaine précision donnée, qui représente la distance entre chaque position représentée par les noeuds. Par exemple, si la largeur de la table est de 10cm et la précision de 2cm, alors il y a 6 points sur l'axe X de $(0, 0)$ à $(10, 0)$ répartis par bonds de 2 horizontalement (donc $\{(0, 0), (2, 0), (4, 0), ..., (10, 0)\}$).

Ensuite, afin de pouvoir facilement « dessiner » les obstacles sur la grille, il fallait pouvoir facilement passer du système de coordonnées de la table (aussi appelé « coordonnées réelles ») au système de coordonnées de la grille. Le système de coordonnées de la grille est représenté par les indices des points de la grille. Ainsi, dans l'exemple précédent, le point réel $(4, 0)$ correspond au point de la grille $(2, 0)$. Puisque les deux repères possèdent la même orientation et le même point d'origine, cela simplifie grandement l'équation 4.1 utilisée pour la transformation de coordonnées du repère A au repère B et ce, sur les axes X et Y.

$$p_A^x = \frac{p_A^x D_B^x}{D_A^x} \qquad p_A^y = \frac{p_A^y D_B^y}{D_A^y} \tag{4.1}$$

Cette équation permet une mise à l'échelle grâce à des dimension de repère D sur les axes X et Y, soit ici les dimensions réelles de la table et la taille de la grille (en nombre de noeuds).

Une fois le système de changement de coordonnées mis en place, des modules de représentation de formes 2D ont été implémentés. La tâche était de pouvoir établir les points appartenant à un cercle ou un rectangle selon les dimensions données et ce, à partir d'une série de valeurs réelles disponibles (générées à l'aide de la précision demandée). L'algorithme se devait d'être le plus efficace possible puisqu'il y a une grande quantité de noeuds à traiter. Les opérations sont donc entièrement calculées à l'aide de la vectorisation avec Numpy. Voici une liste des étapes optimisées effectuées pour déterminer la surface que prend un obstacle sur la grille :

1. Prédéterminer, sur l'ensemble des noeuds de la grille, un espace rectangulaire dans

lequel la surface circulaire se trouve (donc, trouver la boîte coutour du cercle). Il s'agit ici de seulement trouver les points des axes X et Y et non un ensemble de points 2D.

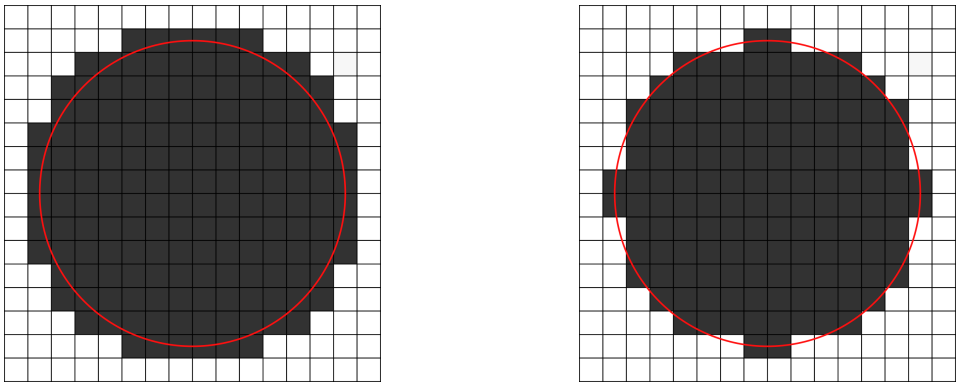
2. Parmis ces points, construire l'ensemble des points 2D possibles (d'après toutes les combinaisons X et Y des points de l'étape 1).
3. Parmis ces points, trouver ceux qui appartiennent au disque de rayon R.
4. Même principe pour les rectangles, mais en remplaçant le rayon par la demi-largeur en X et la demi-hauteur en Y, en omettant l'étape 3 (puisque le rectangle correspond directement à la boîte contour).

La grille devra ainsi contenir des disques représentant chacun des obstacles ainsi que quatres rectangles à aire nulle représentant les côtés de la table. De plus, un *padding* correspondant à la distance maximale entre le centre des roues et l'extrémité du robot devra être ajouté autour de chaque obstacle. Cela permet de réduire les calculs en représentant la position du robot par une simple singularité. Un *padding* de sûreté additionnel peut également être ajouté.

Puisque ce calcul est d'une précision infinie, mais que les points en entrée ont une précision fixe, il n'est pas possible d'utiliser une opération d'égalité. Nous devons ici approximer les calculs en ajoutant la moitiée de la précision p comme *padding* supplémentaire. Cela permet de mimer un arrondi supérieur au lieu d'un arrondi régulier, ce qui permet de couvrir l'entièreté de la forme. Cela revient à légèrement modifier l'équation de base d'un disque :

$$Disque = \left\{(x, y) : (x - c_x)^2 + (y - c_y)^2 \leq (r + 0.5p)^2\right\} \tag{4.2}$$

La figure 4.2 montre bien que sans l'ajout de la moitié de la précision au rayon, l'obstacle n'est pas entièrement couvert, ce qui laisse place à des espaces où le robot pourrait entrer en collision.



(a) Avec ajout de la moitié de la précision (b) Sans ajout de la moitié de la précision

FIGURE 4.2 – Comparaison des couvertures avec ajout de la moitié de la précision au rayon versus sans ajout

b. Champs de force

Afin que les obstacles soient contournés à une certaine distance sans toutefois empêcher une trajectoire possible, nous avons ajouté un « champ de force » émanant des obstacles. En implémentation, cela consiste à ajouter des poids sur chacun des noeuds, en partant d'un poids élevé près des obstacles à un poids presque nul aux extrémitées. Pour ce faire, un module de taux de décroissance a été créé afin de permettre le calcul d'un taux de décroissance selon différents ordres. L'équation 4.3 montre la formule générale selon l'ordre i fourni. La fonction retourne un pourcentage en fonction de la valeur maximale prévue max_x ainsi qu'une valeur quelconque $x \in [0, max_x]$.

$$weight = \left(1 - \left(\frac{x}{m}\right)^i\right) \cdot 100 \tag{4.3}$$

La figure 4.3 montre les taux de décroissances des ordres 1, 2, 3 et 10 (plus la courbe est basse, plus l'ordre est grand), sur une échelle de 0 à 1 afin de simplifier. En pratique, il a été déterminé que l'ordre 3 semblait être le plus optimal, c'est-à-dire le meilleur rapport entre être le plus prudent possible et trouver le chemin le plus court.

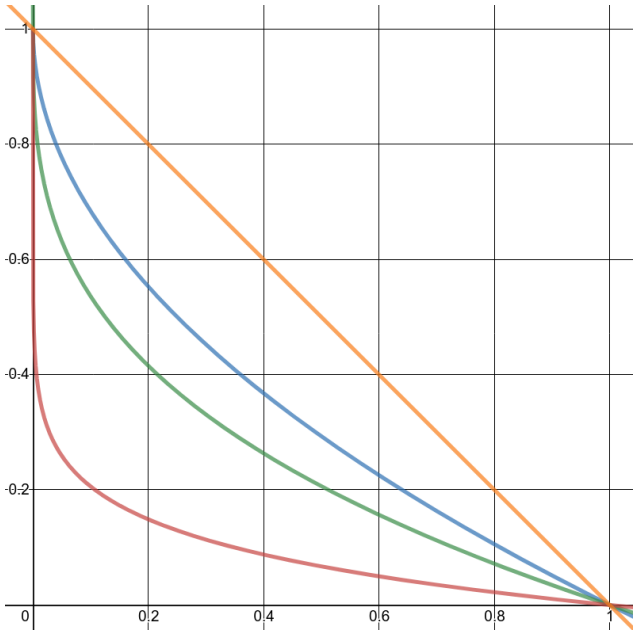


FIGURE 4.3 – Taux de décroissance selon les ordre 1, 2, 3 et 10

c. Algorithme de recherche

Une fois la grille construite, nous devons utiliser un algorithme permettant de trouver le chemin le plus court entre un point de départ et un point d'arrivée. Pour ce faire, nous avons utilisé la librairie `pathfinding`, qui offre un éventail de différents algorithmes, dont Dijkstra et A*. Nous avons jugé cette solution suffisante puisque ces algorithmes de recherche sont très standards et il est donc inutile de les réimplémenter.

La figure 4.4 montre un exemple de grille de recherche et du trajet trouvé.

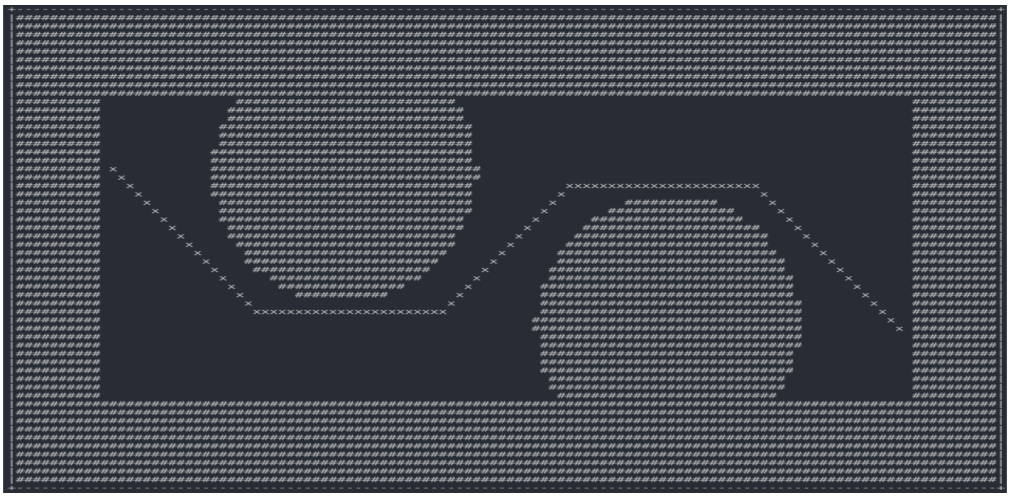


FIGURE 4.4 – Exemple de grille de recherche et chemin trouvé

d. Comparaison des temps d'exécution

La figure 4.5 montre les temps d'exécution, pour différentes précisions de la grille, des étapes de création de la grille et de recherche du meilleur chemin. Notons que les algorithme Dijkstra et A* semblent offrir des performances équivalentes. Notons également qu'une précision entre 1cm et 1.5cm est considérée idéale, alors qu'un temps maximal de 0.5s est visé.

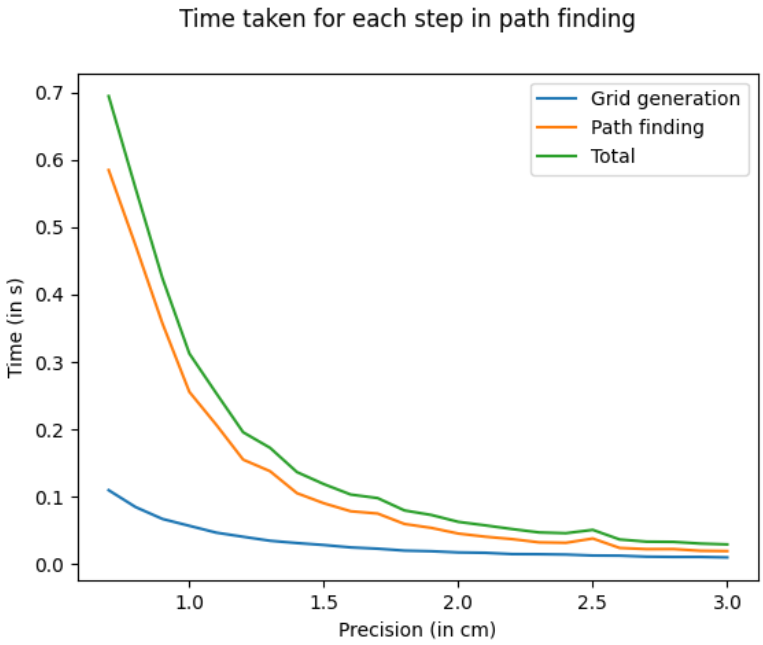


FIGURE 4.5 – Temps d'exécutions des étapes de recherche du meilleur chemin

4.1.1.3 Détection des tableaux de commandes avec lettres

Lors du dernier rapport, la détection des tableaux de commandes avec flèches a été faite. Depuis la dernière remise, nous avons expérimenté avec la détection des panneaux de commandes avec lettres. Nous avons réutilisé le code utilisé pour détecter le panneau de flèches en faisaint des modifications mineures. Ceci a permis de détecter les panneaux de lettres avec une fiabilité raisonnable, mais certainement pas assez élevée afin d'utiliser l'algorithme dans le robot. Comme on peut le voir dans la figure 4.6, des faux positifs existent. Malgré ça, l'algorithme identifie toutes les cases présentant des lettres et est capable d'ignorer ce faux positif afin de sélectionner la région ayant le plus de lettres identifiées. Ces faux positifs peuvent certainement poser problème, mais il faudrait plus d'expérimentation pour le savoir. Ainsi, il faudrait raffiner l'algorithme afin d'ignorer ou d'effacer certaines structures qui peuvent ressembler à des lettres. Il faudrait donc par exemple masquer les flèches qui auraient été détectées précédemment afin de ne pas mélanger le tableau de flèches avec le tableau de lettres.



FIGURE 4.6 – Identification d'un panneau de lettres

4.1.1.4 PCB

La version finale du PCB a été conçue et fabriquée entre le livrable 2 et 3. Celui-ci contient le circuit d'alimentation et de lecture de résistance tel que testé préalablement. De plus, des connecteurs d'interconnexions ont été ajoutés.

Le circuit d'alimentation qui remplace le module acheté a été conçu de façon à ce qu'il soit intégré sur le PCB sans aucun changement aux circuits déjà existants. Il s'agit de l'application typique du TPS565208.

Les schémas des circuits sont disponibles à l'annexe A.

a. Préhenseur

Les modules du préheneur, soit le panier et le préhenseur lui-même, sont tous fontionnels individuellement. Une synchronisation logicielle restait à être effectuée afin de bien intégrer le tout. Le préhenseur est contrôlé par un servomoteur modifié afin de permettre un contrôle en vitesse. Le panier, quant à lui, est contrôlé par un servomoteur contrôlé en position.

Finalement, les matériaux utilisés pour le préhenseur sont les suivants. Le préhenseur est imprimé en 3D en PLA et le panier est en aluminium, des matériaux robustes et fiables.

b. Lecture de puissance

Concernant la lecture de la puissance, nous avons utilisé l'ADC du STM32F4-Discovery couplé avec le ACS7033 mentionné dans la partie 3.1.11.3 afin de pouvoir mesurer la puissance. Il ne restait que les tests finaux du code à effectuer afin de s'assurer de son bon fonctionnement.

De même, la lecture de tension de la batterie permet de facilement prédire la charge restante, puisqu'une chimie acide-plomb est utilisée.

4.1.1.5 Maintien de l'information et diffusion des changements

Afin de faciliter l'intégration des composantes du système, nous avons mis en place une infrastructure permettant de facilement sauvegarder les données et de diffuser les changements d'état à l'ensemble des composantes (interface et robot).

Le maintien des données propres au robot (charge restante, alimentation, status du préhenseur, etc.) se fait dans une classe nommée `RobotInfos`. Dans cette dernière, les informations sont groupées par usage commun. À titre d'exemple, l'ensemble des informations liées au déplacement du robot se trouve sous l'attribut `RobotInfos.movement` qui, en soit, est une instance d'une `dataclass`.

Le partage d'information entre les systèmes se fait à l'aide d'*observers* sur ces attributs. Si `RobotInfos.movement` se fait assigné une nouvelle instance, cela va automatiquement faire appel aux *listeners* attachés à cet attribut. Puisque la station de base et le robot possèdent tous deux une instance de `RobotInfos`, il a suffit d'ajout des *listeners* afin que chaque changement de donnée se propage d'un système à un autre.

Finalement, cette architecture permet en plus de mettre en place une sécurité supplémentaire. Dans le cas où une défaillance surviendrait sur le robot ou que celui-ci redémarrerait, l'ensemble des informations critiques est conservé dans la station de base. Ce dernièr est simplement renvoyé au robot lorsqu'une nouvelle connexion s'établie.

L'ensemble des fonctionnalités précédemment décrites se retrouve également au niveau de la station de base, sous le nom de `GlobalInfos`. Cette classe s'occupe alors de conserver les informations relatives à la séquence, aux mesures prises par le robot, aux trajectoires du robot, etc.

4.1.1.6 Interface

Depuis le dernier rapport, une refonte complète du style a été effectuée. L'interface livrée au livrable 2 n'utilisait pas pleinement le potentiel de SASS, et n'était pas esthétiquement plaisante. Le style a donc été uniformisé à l'aide de l'utilisation de variables de style ainsi qu'une palette de couleurs plus sombres. .

Ensuite, une meilleure division des composantes a été mise en place afin d'optimiser l'espace disponible sur un écran d'ordinateur moyen. De plus, les divisions manquantes au livrable 2 ont été ajoutées. L'interface comporte maintenant 10 divisions principales, chacune contenant des informations nécessaires au suivi du robot et de la séquence d'action. Ces dernières sont présentées et détaillées davantage dans les sections qui suivent.

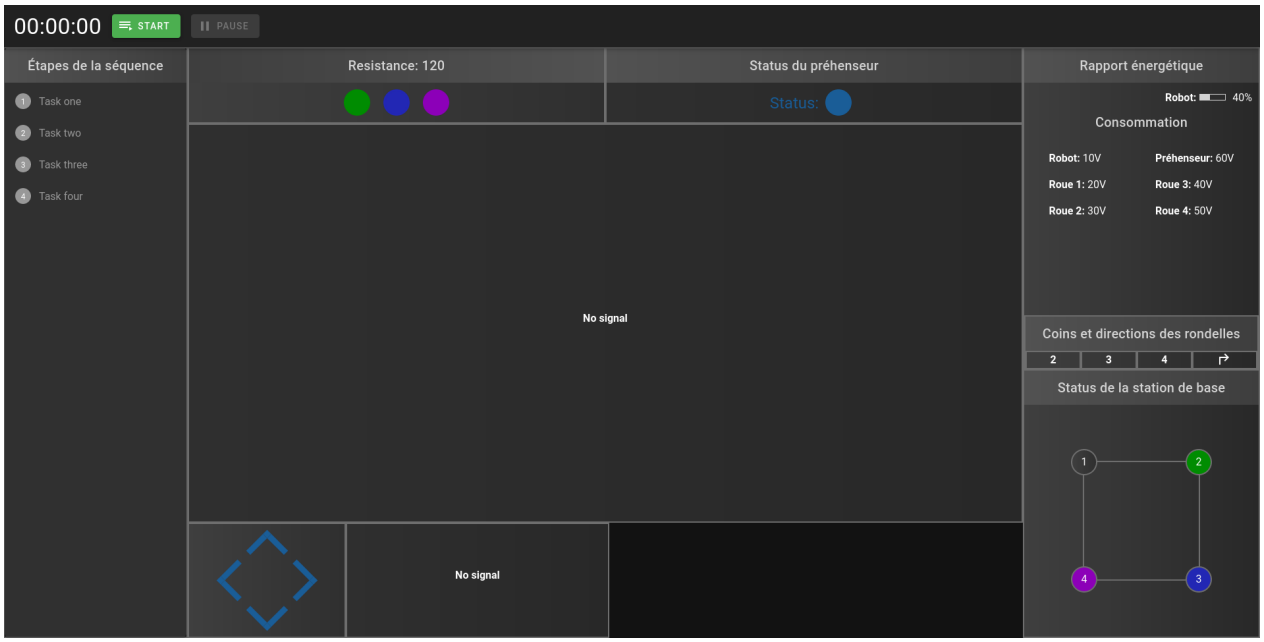


FIGURE 4.7 – Interface améliorée

1) Contrôle du statut de la séquence

La barre du haut indique le temps écoulé depuis le début de la séquence. Elle contient également deux boutons de contrôle : un permettant de démarrer ou stopper la séquence, l'autre permettant de mettre en pause ou de continuer. Ceux-ci réagissent également aux mises à jour envoyées par la station de base, c'est-à-dire lorsque la séquence se termine ou lorsque la séquence est en attente du robot. Ces boutons sont complets et fonctionnels dans l'interface actuelle.

2) Étapes de la séquence

La colonne de gauche présente les étapes de la séquence. Elle indique quelle étape est présentement en cours d'exécution et grise les étapes déjà exécutées. Cet élément est complet et fonctionnel dans l'interface actuelle, c'est-à-dire que la liste des étapes précédentes, de l'étape actuelle et des étapes restantes se met entièrement à jour, en temps réel, selon l'état du robot et de la station de base.

3) Résistance

La première division centrale présente la valeur de la résistance lue à la station de résistance et les couleurs des rondelles équivalentes au code de couleur de cette valeur. Cet élément est présentement défini manuellement et n'est pas mis à jour selon l'état de la station de base.

4) Préhenseur

La deuxième division centrale indique le statut du préhenseur. Un rond de couleur s'allume lorsque le préhenseur tient une rondelle, et s'éteint lorsqu'il la lâche. Cet élément est présentement défini manuellement et n'est pas mis à jour selon l'état de la station de base.

5) Caméra monde

Cette division aurait dû contenir l'image en temps réel de la caméra monde, sur laquelle la trajectoire prévue, la trajectoire réelle parcourue ainsi que les composantes de la table (obstacles, murs, rondelles, etc.) auraient été dessinées. Cet élément est donc présentement manquant dans l'interface actuelle.

6) Contrôle manuel du robot

Cette division, utile pour tester le robot en dehors de la séquence, permettait de déplacer le robot en utilisant les flèches. Cet élément est complet et fonctionnel dans l'interface actuelle.

7) Caméra robot

Cette division aurait dû contenir l'image en temps réel de la caméra du robot. Cet élément est donc manquant dans l'interface actuelle.

8) Rapport énergétique (nouveau du livrable 3)

Cette division contient la consommation énergétique des différentes parties du robot (chacune de ses roues, le préhenseur et le robot au complet) en plus de l'état de sa batterie, et le temps restant de la batterie. Cet élément est présentement défini manuellement et n'est pas mis à jour selon l'état de la station de base.

9) Position finale des rondelles (nouveau du livrable 3)

Cette division indique la position finale que devront avoir les rondelles, c'est-à-dire sur quel coin elles devront être posées, et dans quelle direction. Cet élément est présentement défini manuellement et n'est pas mis à jour selon l'état de la station de base.

10) État de la station de base (nouveau du livrable 3)

Cette division indique quelles rondelles ont été déposées dans la zone cible. Lorsque la rondelle est à la position voulue, le coin où elle a été déposée devient la couleur de la rondelle correspondante. Cet élément est présentement défini manuellement et n'est pas mis à jour selon l'état de la station de base.

4.1.2 Éléments manquants

4.1.2.1 Détection du robot

Le système de détection de la position ainsi que de l'angle de rotation du robot n'a pas encore été implémenté. Par contre, son fonctionnement sera pratiquement le même que celui de la détection des obstacles, à deux éléments près.

1. Il a été planifié d'utiliser des `ArucoBoard` (ou des `CharucoBoard`) afin d'augmenter la précision de détection ainsi que de permettre la détection même lorsqu'une partie du code `Aruco` est non-détectable (image floue, robot hors du cadre de l'image, etc.).
2. La vecteur de rotation 3D retoruné par `OpenCV` devra être utilisé et ne pourra pas être mis à zéro puisque nous devons mesurer l'angle de rotation du robot.

De plus, les modules du programmes ont déjà été pensés afin de permettre une intégration facile et rapide pour la détection de n'importe quel objet utilisant les marqueurs `Aruco` (ou l'un de ses dérivés).

Finalement, ce manquement englobe également l'ensemble des fonctionnalités qui en sont dérivées, comme l'enregistrement de la trajectoire réelle effectuée par le robot ainsi que son envoie à l'interface.

4.1.2.2 Système de changement de coordonnées complet

Le système de changement de coordonnées mis en place avec le pathfinding n'est pas mature. Il ne prend pas encore en compte les translations et rotations nécessaires lors d'un changement de repère. Cet ajout sera nécessaire afin de passer d'un repère image (plusieurs formats possibles) vers le repère table. Par contre, cet ajout est plutôt minime et serait facile à implémenter avec le système actuel.

4.1.2.3 Pathfinding - *padding* ajusté à la rotation du robot

L'algorithme d'ajout de *padding* pour le pathfinding ne prend pas en compte l'orientation du robot. Puisque celui-ci risque fort bien de tourner en même temps que d'avancer et qu'il n'est pas exactement carré, le rayon limite du robot n'est pas le même dans toutes les directions. Ainsi, il se pourrait que le *padding* doive être plus petit selon certains angles, et

plus grand selon d'autres, ce qui permettrait au robot de passer à des endroits qui, pour l'instant, sont inaccessibles. Par contre, cet ajout ne devrait pas demander beaucoup d'efforts supplémentaires. Il suffirait de remplacer la formule d'un cercle par celle d'une ellipse pour les obstacles circulaires (très similaire) et de déterminer les *paddings* en X et Y pour les obstacles rectangulaires.

4.1.2.4 Connexion de l'interface à la station de base

La station de base à été implémentée de sorte à ce que chaque changement de l'état interne de la station (ex : position du robot, résistances mesurées, charge restante, etc.) envoie un événement à l'interface avec les nouvelles informations. Il avait été planifié, pour la troisième remise, de finaliser la gestion de ces événements au niveau de l'interface afin de mettre à jour les informations affichées. De plus, l'interface contient déjà l'ensemble des composantes nécessaires afin d'afficher toutes les informations requises. Cet ajout se serait donc fait facilement.

4.1.2.5 Détection des tableaux de commandes

Pour la détection de tableaux, il aurait fallu faire plus de tests afin de savoir si les algorithmes performent bien dans les conditions réelles d'une compétition. De plus, il aurait été important d'intégrer aux algorithmes une façon de détecter s'ils ne donnent pas de bons résultats afin de replacer le robot et de recommencer l'algorithme avec une nouvelle image.

4.1.2.6 PCB

Le PCB venait d'être reçu quand l'université a été fermée. Il ne restait alors que l'assemblage et les tests finaux. Tous les circuits avaient déjà été testés séparément et il ne restait qu'à valider que leur implantation sur le PCB final était correcte. Ceci inclue l'alimentation, le circuit de lecture de la résistance, de l'électroaimant ainsi que les lectures de puissance. Les schémas des circuits sont présentés à l'annexe A.

4.1.2.7 Préhenseur

Pour le phésenseur, il ne restait que l'installation finale physique sur le robot et l'ajustement de la distance entre l'aimant et la rondelle à faire. Le montage physique attendu est présenté à l'annexe B. L'ajout de feuilles de papier ou plastique entre l'aimant et la rondelle avait pour but de plus facilement ajuster la force. Du côté de l'intégration, il ne restait qu'à valider que la partie logicielle était en mesure de faire atteindre les positions voulues au préhenseur, puisque cela n'avait été testé qu'avec un logiciel séparé.

Aussi, il avait deux méthodes envisagées afin de détecter la présence d'une rondelle sur le préhenseur. Soit par la mesure du courant consommé par le servomoteur, ou bien par la présence d'une diode photoélectrique entre l'électroaiment et la rondelle. Ces méthodes n'ont pas étés testées, mais l'équipe envisageait d'opter pour la méthode de lecture de courant.

4.1.2.8 Code de vérification pour communication UART

Afin d'établir une communication UART fiable entre le RaspberryPi et le microcontrôleur STM32, nous pensions ajouter à notre implémentation un code de vérification CRC afin de détecter toute corruption de paquets. Il était prévu de le faire directement après que les fonctionnalités logicielles de lecture de puissance et de contrôle du préhenseur aient été testées et validées. Malheureusement, dû à la fermeture de l'universtié, cette fonctionnalité n'a pas pu être ajoutée au projet, mais il n'aurait pas été trop difficile de le faire et n'aurait pas ralenti de façon significative la communication entre les deux composants.

4.1.3 Points forts

4.1.3.1 Intégration rapide grâce à l'injection automatisée

En raison de l'utilisation d'un système d'injection automatisé, le projet ne requiert pas la création d'un script interminable afin de créer et lier les dépendances entre chaque module. Ainsi, outre les dépendances de base (regroupant entre autres les implémentations technologiques), lors de l'intégration d'un module, il suffit de s'assurer de bien définir les types des dépendances et la liaison s'effectue automatiquement. De plus, ce système s'avère particulièrement utile lorsqu'une dépendance doit être ajoutée ou modifiée, puisque nous n'avons pas besoin de redéfinir l'ordre de création de chaque classe. Finalement, l'injection de dépendances automatisée permet de facilement implémenter et utiliser le principe d'inversion de dépendances, qui s'avère utile afin de développer une architecture modulaire, flexible et facilement testable.

4.1.3.2 Simplicité du système de séquençage

Le système de séquençage a été développé dans l'intention d'être le plus générique possible. Ceci dit, deux niveaux d'abstractions sont présents : la tâche (unité de travail) ainsi que son conteneur. Un conteneur de tâches contient une liste de tâches et possède un comportement générique pour l'exécution séquentielle de chacune d'elle. Selon le type d'exception soulevé, le conteneur agira en conséquence en retentant l'exécution de la tâche, en continuant vers la prochaine tâche, en attendant une action particulière avant de continuer ou encore en arrêtant la séquence au complet. Chaque implémentation du conteneur décide d'ajouter ses propres tâches dans la liste, en ordre d'exécution. Cela permet de choisir différent conteneur au moment de l'exécution du programme afin de tester différentes séquences (la séquence principale étant celle demandée par l'énoncé). Ensuite, chaque tâche possède également un comportement de base. On peut définir leur exécution comme étant non-déterministe et ainsi demander de réessayer si un problème est survenu. C'est à chaque tâche de gérer ses exceptions et quand demander un réessaie ou quand se mettre en attente du robot ou de l'interface. Chacune d'elle peut également interagir librement avec les différents modules de détection d'obstacles, de planification de trajectoire ou de contrôle du robot.

4.1.3.3 Architecture bien définie

L'architecture mise en place tente de suivre le plus fidèlement possible le principe du « Clean architecture », c'est-à-dire une architecture définie par couches. Ces différentes couches ont la permission ou non d'intéragir avec d'autres couches et chacune a une responsabilité propre. Cela permet de plus facilement comprendre les intéractions entre les modules du système tout en forçant le développement de modules découplés et responsabilisés. Par exemple, la couche du « domaine » est celle qui contient l'ensemble de la logique et ne peut communiquer qu'avec les interfaces des modules implémentés dans des couches plus externes. D'autre part, la couche « infrastructure » contient les implémentations des différents modules interchangeables tels que les choix technologiques ou les communications externes. Les éléments de cette couches ne peuvent pas communiquer avec d'autres couches afin d'assurer leur interchangeabilité. La principale exception amenée au projet par souci de simplicité est que la couche « application » est manquante, puisque sa compréhension et son utilisation pourraient amener plusieurs débats au sein de l'équipe. Il a donc été jugé préférable d'utiliser directement les modules du domaine.

4.1.3.4 Architecture du style de l'interface

Nous avons utilisé SASS (SCSS) pour le style, ce qui permet d'utiliser des variables globales et des feuilles de style s'appliquant sur l'entièreté du projet. Nous avons donc une excellente séparation entre les différentes composantes sur cet aspect.

4.1.3.5 Pathfinding - création de la grille très efficace

Comme présenté à la figure 4.5, l'algorithme de création de la grille demeure extrêmement efficace, même avec une distance de précision très faible. Cette étape était supposé être le goulot d'étranglement de l'ensemble de l'algorithme. Nous sommes donc très fiers de sa performance.

4.1.4 Points faibles

4.1.4.1 Manque de tests automatisés

Malheureusement, de nombreux tests automatisés sont manquants dans l'application pour diverses raisons. Tout d'abord, l'équipe logicielle a subit quelques manques de main d'oeuvre durant la session, ce qui a demandé de mettre plus d'importance sur le développement de nouveaux systèmes au détrimant des tests afin de réussir à rester dans les temps. Ensuite, la majorité des tests automatisés demandent la récolte de grandes quantités de données, telles que des prises de mesures, de photos et de vidéos. Pour les mêmes raisons que celles présentées précédemment, il a été jugé que les bénéfices soutirés de telles récoltes de données étaient plus faibles que ceux soutirés du développement de nouvelles fonctionalités. Finalement, de nombreux modules sont plutôt difficiles à tester de façon unitaire en raison de leurs dépendances à des données physiques (tel que le robot) ou encore à des librairies multimédia

(tel que OpenCV). Ces dépendances demandent beaucoup de temps et de ressources afin d'être « stubbé » pour les tests automatisés.

Néanmoins, il est à noter que l'ensemble des modules ont tout de même été dûment testés lors de leur développement. Les tests automatisés ne permettent que de s'assurer qu'un changement apporté à une fonctionnalité n'empêche pas la réalisation d'une autre. De plus, nous sommes pleinement conscients de leur importance dans le domaine du logiciel. La seule raison pour laquelle ils n'ont pas été développés est que des tâches plus prioritaires étaient présentes. Par contre, leur développement était prévu aux alentours de la fin du livrable 3 et du début du librable 4.

4.2 Gestion des finances

4.2.1 Liste des achats

Le design des circuits et le choix de la batterie ont fait en sorte que le coût des achats totaux est très bas comparativement au budget alloué. Cela s'explique par l'utilisation d'un circuit de lecture de résistance très simple, la présence d'un seul convertisseur de tension et l'utilisation d'une batterie au plomb. La liste des achats effectués est présentée au tableau 4.1.

Achat	Coût [$]
Batterie acide-plomb	30,00
PCB	34,00
Impression préhenseur	15,00
Pièces panier à rondelle	5,00
IC Capteurs de courant x 6	20,00
Headers male-femelle	10,00
Condensateurs 0603	1,85
Résistances 0603	3,85
Mosfet de puissance	2,90
Mosfets LED et lecture resistance	1,40
Amplificateur opérationnel	1,75
Servomoteurs x 2	10,00
Électroaimant	15,00
Convertisseur	8,46
STM32	42,00
Total	201,21

TABLEAU 4.1 – Liste des achats effectués pour la conception du projet

4.2.2 Projection des dépenses

La majorité des achats déjà effectués, il ne restait que des petits détails à régler afin d'obtenir la liste complète des achats pour un système fonctionnel, plus précisément, des connecteurs ainsi que des headers afin de rendre le tout propre. Le total restant est estimé à un maximum de 20,00$.

4.3 Gestion des ressources humaines

4.3.1 Outils et protocoles de communication

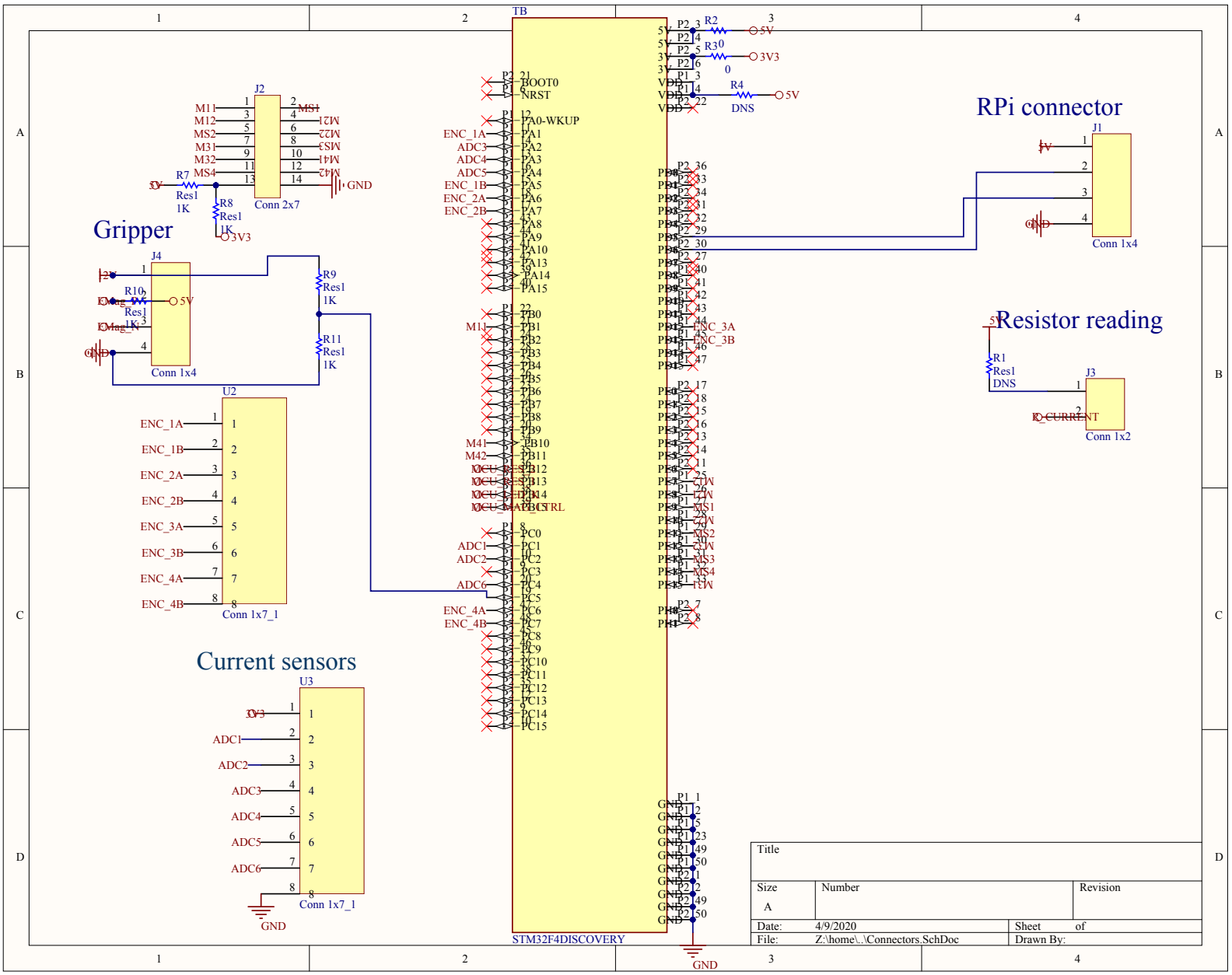
Afin d'établir une communication écrite entre les membres de l'équipes, la plateforme de communication *Slack* a été utilisée. Des chaînes respectives ont été crées pour chacune des disciplines du projet (logicielle, électronique et embarquée). Pour les messages moins formels, nous utilisions l'application *Messenger* de *Facebook*. Dans les deux cas, les échanges entre les membres étaient assez fréquents. Nous nous tenions au courant des différents progrès qui ont été faits ainsi que des tâches qui restaient à accomplir. Finalement, l'application *Discord* a été utilisée afin d'effectuer des appels vocaux et vidéos. Cela a permis d'effectuer des réunions à distance (plus particulièrement suite à la fermeture du campus universitaire), de travailler en équipe et en temps réel, ainsi que de prendre des décisions en groupe.

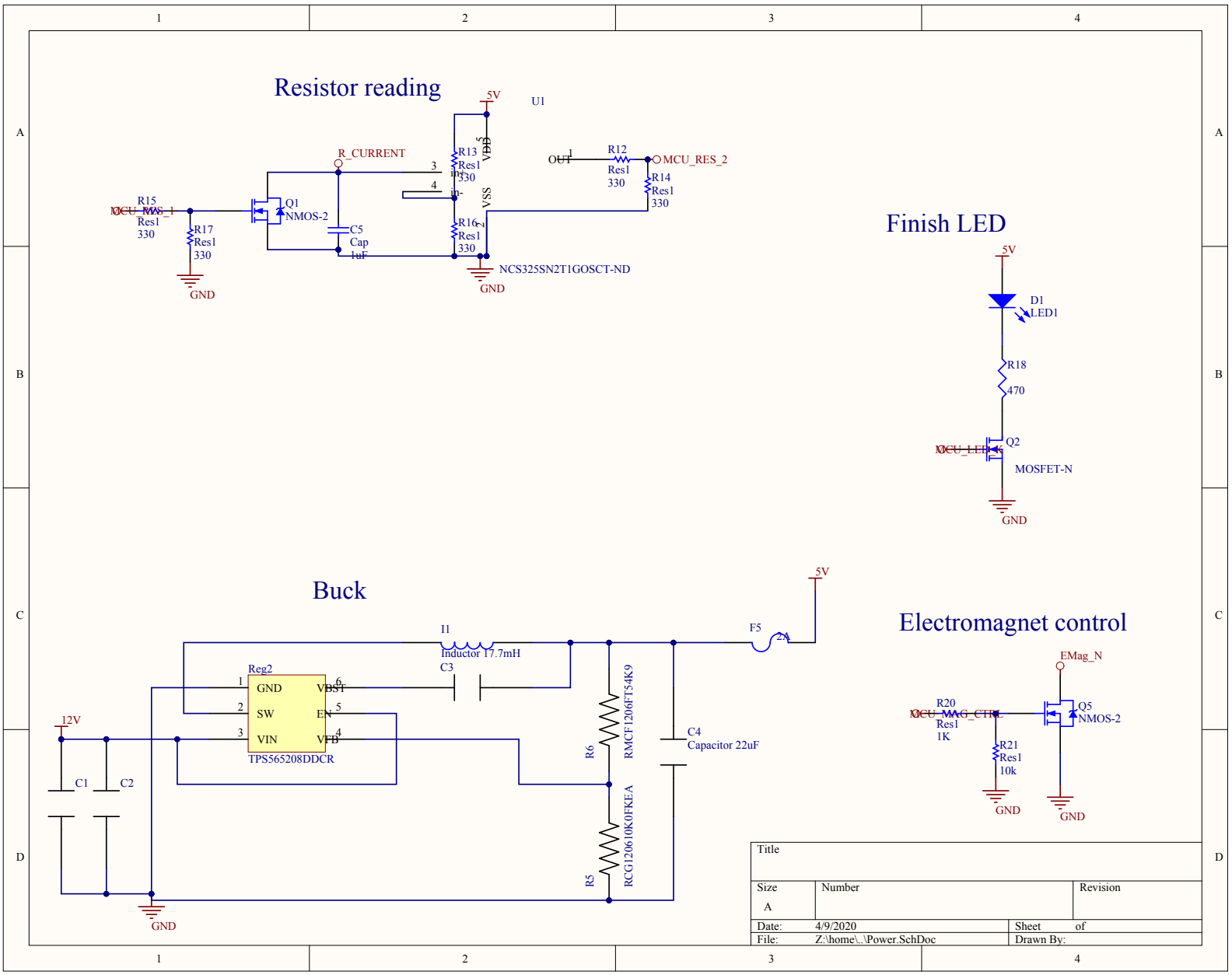
4.3.2 Critiques sur les méthodes de fonctionnement de l'équipe

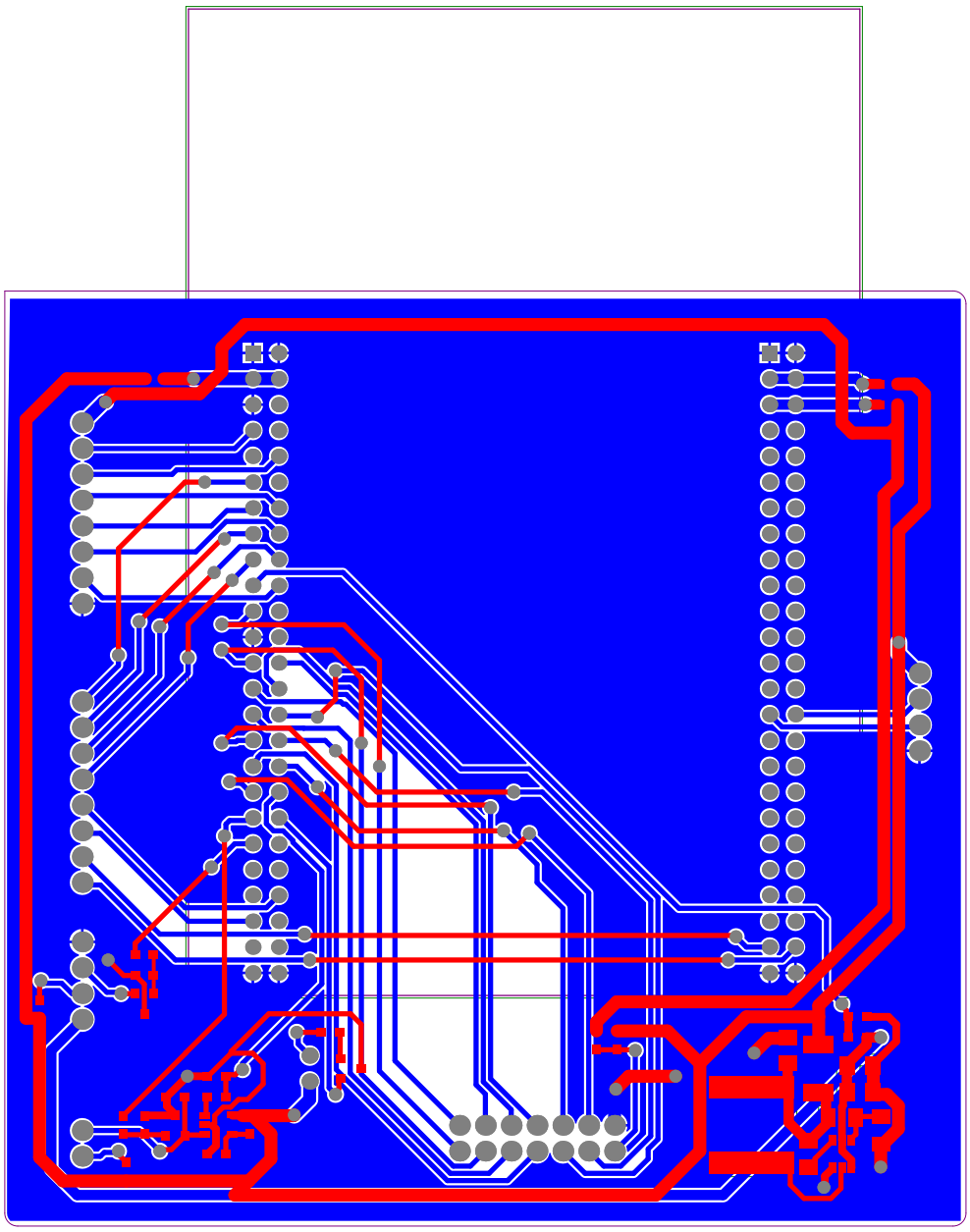
Au début de la session, les tâches étaient distribuées à l'aide de l'outil *Github Projects* et leurs niveaux d'avancement étaient discutés lors des réunions (souvent à distance). Par contre, le projet n'avançait pas par rapport à la vitesse voulue en raison d'un manque de communication entre les membres de l'équipe. Afin de remédier à la situation, nous avons commencé à tous travailler dans la même pièce, ce qui a grandement faciliter la coopération. De plus, les problèmes étaient réglés plus rapidement puisque les questions pouvaient être répondues en temps réel.

Annexe A

Plans PCB


TB
STM32F4DISCOVERY
RPi connector
Gripper
Resistor reading
Current sensors
J1
Conn 1x4
J2
Conn 2x7
J3
Conn 1x2
J4
Conn 1x4
U2
Conn 1x7_1
U3
Conn 1x7_1
R1
R2
R3
R4
R7
R8
R9
R10
R11
Res1
DNS
0
1K
5V
3V3
GND
BOOT0
NRST
PA0-WKUP
ENC_1A
ENC_1B
ENC_2A
ENC_2B
ENC_3A
ENC_3B
ENC_4A
ENC_4B
ADC1
ADC2
ADC3
ADC4
ADC5
ADC6
M11
M12
M21
M22
M31
M32
M41
M42
MS1
MS2
MS3
MS4
CURRENT
Title
Size
A
Number
Revision
Date:
4/9/2020
Sheet
of
File:
Z:\home\..\Connectors.SchDoc
Drawn By:


Resistor reading
5V
U1
R_CURRENT
R13
Res1
330
VDD
OUT
R12
Res1
330
MCU_RES_2
R14
Res1
330
R15
MCU_MAS_1
Res1
330
Q1
NMOS-2
R17
Res1
330
C5
Cap
1uF
R16
Res1
330
VSS
NCS325SN2T1GOSCT-ND
GND
GND
Finish LED
5V
D1
LED1
R18
470
Q2
MCU_LED_1
MOSFET-N
GND
Buck
5V
I1
Inductor 17.7mH
F5
2A
C3
Reg2
GND
SW
VIN
VBST
EN
VFB
TPS565208DDCR
12V
C1
C2
R6
RMCF1206FT54K9
C4
Capacitor 22uF
R5
RCG120610K0FKEA
GND
Electromagnet control
EMag_N
R20
MCU_MAG_CTRL
Res1
1K
Q5
NMOS-2
R21
Res1
10k
GND
GND
Title
Size
A
Number
Revision
Date:
4/9/2020
Sheet
of
File:
Z:\home\..\Power.SchDoc
Drawn By:



Annexe B

Montage préhenseur

